BMC-K4510

USER'S AND PROGRAMMER'S GUIDE

a fantasy 8/16-bit computer

Michael Longval
project orchestrator

Draft version: alpha-0.3+71
CPU 45GS10 @ optimal MHz · VICKY · 4 reSIDs · 256 MB RAM
Date: 27.08.2026

This is alpha documentation. It describes a machine that is still being built, and it is written alongside it. Everything in it — commands, keys, registers, the layout of files, the names of things — is subject to change, and some of it will have changed between this printing and the machine you are running. Where the two disagree, the machine is right and this book is behind it; Appendix B says how to tell us. The version on the cover is the build this book was made from.

Contents

Part I

User’s Guide

Chapter 1

The Machine

The BMC-K4510 is a fantasy computer: a machine that never existed, built the way 1985 might have built it with no budget committee in the room. It is not an emulation of any real computer. Its CPU, video chip, sound, and operating software are its own; the parts it borrows — a 6502-family instruction set, the SID sound chip — it borrows openly and then outgrows.

1.1 What is the BMC-K4510?

- **CPU: the 45GS10** — the MEGA65's 45GS02 instruction set (a 4510 with the Q pseudo-register and 32-bit flat addressing) plus this machine's own memory management: bank registers, a far-call gate, and RAM under the ROM. It runs *at optimal MHz*: the machine is a fantasy, its timings are suggestions, and the clock is whatever the host it runs on can hold at sixty frames a second with no gaps in the sound. A desktop of the last ten years holds a hundred megahertz or more; a Raspberry Pi 3B+ holds fifteen. Today the clock is a setting (F7, under Machine — the desktop starts at 40.5 MHz, the MEGA65's number, and the Pi at 15) — but the first time the machine boots on a host it measures that host, a third of a second with the SIDs sounding, and sets the clock to the highest step it can hold with room to spare. The answer is kept, so later boots pay nothing; a clock you choose yourself in the menu is never second-guessed; and while a program is running, the machine keeps an eye on how much of each frame it is taking, stepping the clock down if it is losing. What a clock costs depends on what the program does with it — an arithmetic loop and a loop polling the raster differ by three to one — so the measurement is a fair guess rather than a promise, and `BENCH` is there for when you want the whole picture. `INFO` reports the clock in force, read live, which is the only honest number — and the banner

names none, because the machine no longer has one speed.

- **Memory: 256 MB**, flat, 28-bit. The CPU sees 64 KB at a time; everything else is one instruction away.
- **Video: VICKY** — 640×480, 256 colours from a 24-bit palette, four layers, 128 sprites, a blitter that draws lines and filled triangles, and a display-list coprocessor named SHEILA. Smaller modes (640×240, 320×240, 320×200, 160×200) are drawn into the same glass, doubled and centred, so the raster is always 480 lines however few of them the picture uses.
- **Sound: four SID chips** (reSID 6581) and a floating-point MATH unit the BASIC leans on.
- **A system ROM**: a shell with directories, a Wozmon-style monitor, and EhBASIC 2.22 extended with graphics and sound — with three more languages one word away: BBC BASIC on the Tube, Forth, and CP/M.

1.2 Building the emulator

On the **desktop** (Linux), three lines build the whole machine:

```
git clone https://github.com/mlongval/bmc-k4510
cd bmc-k4510 && ./setup.sh
./k4510
```

`setup.sh` installs the compilers and SDL2, builds everything, and runs the machine's ten test suites.

Or: putting it on a Raspberry Pi

On the **Raspberry Pi 3B+** nothing is built — download the SD-card zip from the project's releases page and put it on a card — either unzip it onto the card's root yourself, or let the script do it:

```
./install-sd.sh bmc-k4510-pi3.zip /media/you/CARD
```

Use the official power supply — 5.1 V, 2.5 A. A phone charger or a thin cable sags with three cores flat out; the firmware then caps the ARM clock and the machine runs at a fraction of its speed while looking exactly like a software fault. That cost this project a night: the same card went from 18 to 55 frames a second on the right supply. `INFO` and `BENCH` say when it is happening. (The BMC64 keyboard adapter takes positive-centre barrel power; on a micro-USB supply, a USB keyboard is the way in.)

Use an SDHC card — 4 to 32 GB, specifically. SDHC ships FAT32 from the factory and just works. Older plain SD cards (2 GB and under) do *not* boot — we tested. Bigger SDXC cards (64 GB+) ship exFAT and will not boot until reformatted; `./install-sd.sh zip /dev/sdX --format` does that (and erases the card). The machine needs about 5 MB, so the smallest SDHC card sold is plenty.

1.3 First boot

Switch it on (on the desktop: `./k4510`; on the Raspberry Pi 3B+: insert the card and power up). The machine greets you in yellow on blue:

The `/]` at the bottom is the shell prompt — the `/` is the directory you are in. Type `HELP` for the commands, `INFO` for the machine's full self-description, and `DIR` to see your files.

Keys on the desktop: Escape is RUN/STOP (stops a BASIC program), Ctrl-C is STOP, Shift+Escape quits the emulator. Reset is a chord, the way it was Commodore+Restore on the C64: Super+PageUp (the Windows or Command key and PageUp) — one key cannot do it by accident. F7 opens the menu.

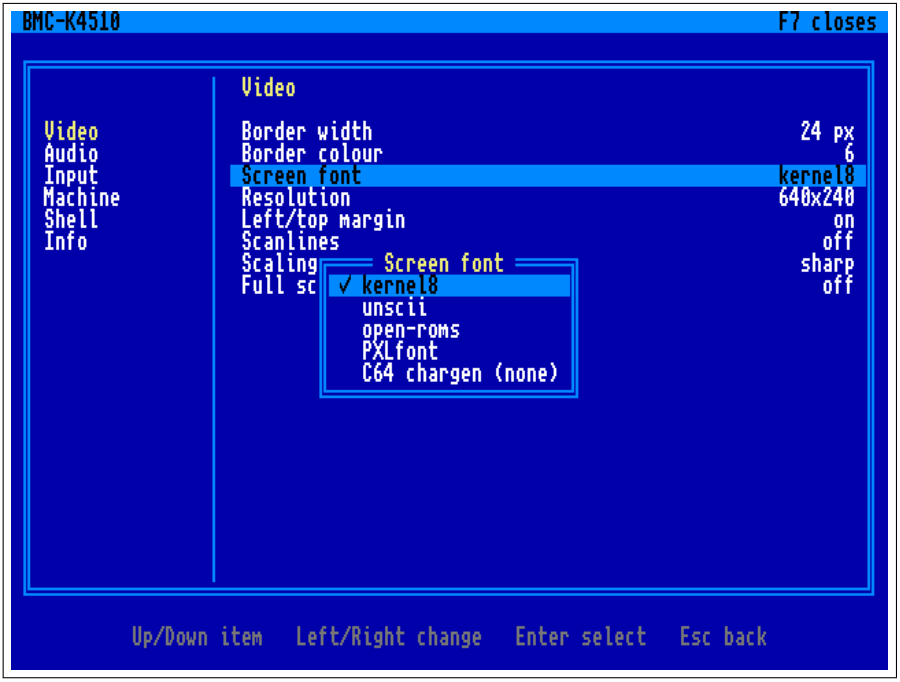
1.4 The F7 menu

F7 (unshifted; Shift+F7 still reaches BBC BASIC) freezes the machine and takes the whole screen, in the manner of the C64 Ultimate's: the categories down the left, the settings of the chosen one on the right,

The boot screen: five stepped colour bars, then the machine, its CPU, its memory and its chips — and no clock, because the machine no longer has one speed. Everything else, the clock in force included, is one `INFO` away, and `LOGO` prints this again.

and a line at the foot saying what the keys do in whichever pane holds the cursor. Sound stops, and closing the menu resumes exactly where it froze — the machine is not disturbed, only hidden. The menu draws with its own font and never touches the machine's memory, so it opens even when a program has wrecked the screen, and being opaque it reads the same whatever was there.

Up and Down move; Enter or Right crosses from the categories to the settings; Left and Right step a value; Escape goes back a pane, and closes at the categories; F7 closes from anywhere. A setting with a list of choices opens that list over the panes, and *applies as the cursor passes each one* — the screen font swaps under the menu so you can see it — with Escape putting back whatever was there when the list opened.



The F7 menu: categories on the left, the Video page on the right, the screen-font list open over them.

Video border width and colour around the picture; the screen font (*kernel8*, *unscii*, *open-roms*, *PXLfont* — a live swap of the chargen at `$010000`, the two PETSCII fonts rearranged into ASCII order, and *C64 chargen* if you have put a Commodore character ROM in `/SYSTEM/chargen.bin` — the entry says so when you have not); *scanlines*, a dark line between each of the machine's, in three strengths — the picture is drawn at twice the height and every second row dimmed, so it is exact rather than an overlay, and it costs about a third of a millisecond a frame; *scaling*, which is how the picture is fitted to the window: *sharp* (hard pixels, any window size — so at a size that is not a whole multiple some pixel rows are wider than others), *soft* (smoothed), or *sharp-fit* (hard pixels *and* a whole-number multiple, so every pixel of the machine is the same size on the glass and what does not divide becomes a black border);

full screen.

The figures in this book are taken with the effects off, whatever the settings say, so that what you see here is the machine's own output.

Audio volume.

Input the reset chord (Super, Ctrl or Alt + PageUp, or Ctrl+Alt+Del); which key opens the menu (F7, F8, F11 or Pause).

Machine *save state* and *load state*, four slots each (the whole machine — CPU, every used page of the 256 MB, the banks and MAP, VICKY, the devices, JIM — to `k4510-slotN.k4s` beside the settings file; the row shows when the slot was written; the Tube co-processor is not in the file and is stopped by a load); *CPU clock* — ten steps from 202.5 MHz down to 10, live (202.5, 162, 121.5, 81, 60, 40.5, 30, 20, 15, 10), measured for this host at its first boot; choosing one here switches *Auto clock* off, because a clock chosen by hand is not to be second-guessed. *Auto clock* — on, the machine measures the host at boot and steps down if the sound starves; off, the clock is whatever the row above says. The Info page names what the host measured. A clock is right for a host when it holds 60 frames a second with no gaps in the sound; above that the programs run faster than the host keeps up with, the frame rate falls and the music slows with it. `BENCH` measures every step (Chapter 2); reset; power cycle (memory cleared, ROM reloaded, back at / with `STARTUP.BAT` run again); stop the Tube co-processor; quit — *Power off* on the Pi.

Shell whether an unknown word at the prompt may run a CP/M `.COM` from drive A: (Chapter 6). Off to begin with, on purpose: `D` typed for `DIR` should not start a Z80 program. It changes only the guess — a command that names `CPM` outright, an alias among them, works either way. This is the first setting the machine itself reads: the host puts the menu's switches in `$D521` and the ROM looks there.

Info version, ROM, files, host.

Leaving the menu writes the settings to `k4510.cfg` beside `fs/` (on the Pi, `SD:/k4510/k4510.cfg`) — a plain `key = value` file you may edit; comments and keys it does not know are kept. Adding a setting is one row in `core/ui/settings.c`, one row in the menu tree in `core/ui/menu.c`, and the place that reads it; the same code runs on the Pi, where the C64 keyboard's F7 opens it too.

Chapter 2
The Shell

The shell is what the machine boots into. It is a file manager, a program launcher and a front door to the monitor, and every command here also works from BASIC with a * in front. Type HELP for the whole command set on one screen — the text it prints is itself a file on disk, a hidden one called /.HELP.

2.1 Files and directories

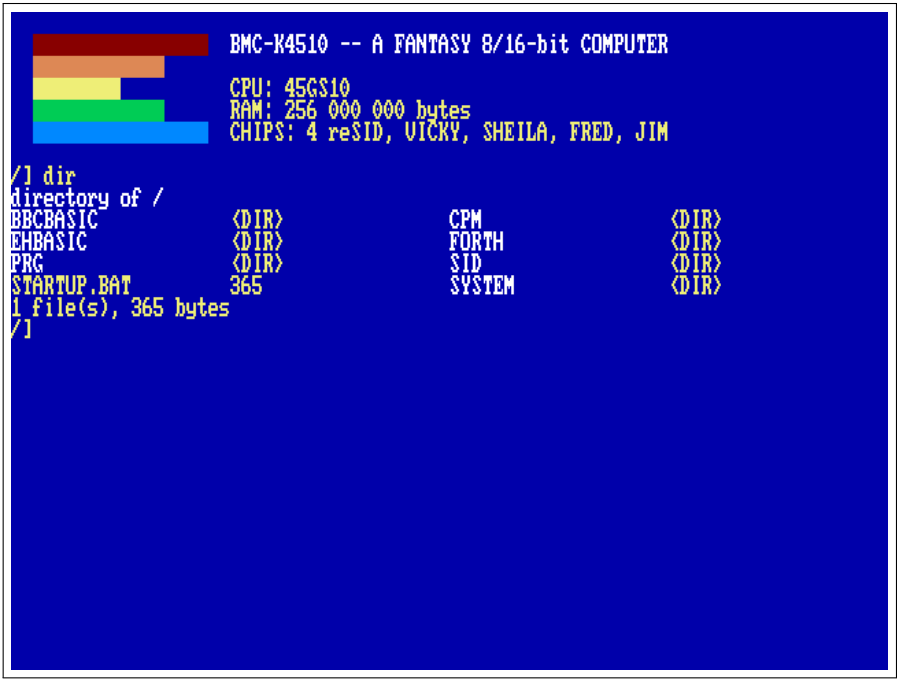
Files live on the host: the fs/ directory beside the emulator on the desktop, the k4510/fs/ folder of the SD card on the Pi. The machine sees that directory as / and nothing above it. Inside, one directory per language, and a few that are the machine's own:

/PRG	machine-code programs: the demos, EDIT and VI, TELNET, BUG
/EHBASIC	EhBASIC programs
/BBCBASIC	BBC BASIC programs, for the Tube
/FORTH	Forth
/CPM	the Z80's drives, A to P, one folder each (Chapter 6)
/SID	music for SIDPLAY — a link to your own HVSC copy, not shipped
/SYSTEM	the machine's own: STARTUP.SAMPLE, BUGREPORTS/, a chargen.bin if you add one
/STARTUP.BAT	yours; run at power-on (Section 2.10)

A bare name not found where you are is looked for in /PRG, /EHBASIC, /BBCBASIC and /FORTH as well, so you rarely need a path; a name is matched regardless of case when the exact one is absent; and .. never climbs above /.

```
DIR [A] [dir | pattern]     LS is the same
CD dir       CD ..          CHDIR is the same
MKDIR dir    RMDIR dir
RM name                     DEL and ERASE are the same
RENAME old new              REN and MV are the same
CP from to                  a file (COPY is memory: see Housekeeping)
TYPE name                   a screenful at a time; q stops
XD name                     a hex dump (HEX is the same); Esc stops it
LOAD name [addr]            SAVE name from.to
EDIT name    VI name        the editors, Chapter 8
```

Names that begin with a dot are hidden; `DIR A` shows them. `DIR` also takes a directory to look in, or a pattern to match: `DIR PRG` lists that folder without going there, `DIR *.PAS` and `DIR ?.TXT` match here (* any run of characters, ? any one), and `DIR A *.BAS` means both at once.



`DIR`: directories first in white, then files with their sizes, two columns.

2.2 Running things

`RUN balls.prg` runs a program; so does `RUN balls`, and so does plain `BALLS` — an unknown word is tried as a program on disk before the shell gives up, *with its arguments*, the way REXX did it on the mainframes. A program reads its arguments through the `ARGS` system call; `SAY` is the demonstration:

```
SAY HELLO FROM THE DISK
```

prints `HELLO FROM THE DISK` — there is no `SAY` command, only a `say.prg` in `/PRG`. Try `SIDPLAY`, the SID jukebox; `EDIT` and `VI` are the editors (Chapter 8); and four words open whole languages: `EHBASIC` (Chapter 3), `BBC` (Chapter 4), `FORTH` (Chapter 5) and `CPM` (Chapter 6).

2.3 SWAP: running one thing from inside another

A program is loaded where the last one was, so starting one from inside a BASIC would ordinarily flatten the program you were writing. `SWAP command` puts the caller away first — the whole of the CPU's memory, and the screen — runs the command on a clean machine, and gives the caller back untouched:

```
*SWAP EDIT MYPROG.BAS
```

from EhBASIC edits a file and returns to your program, variables and screen as they were. One level deep; the thing being run must be an ordinary program, and a BASIC's far memory is not disturbed because nothing else touches it. BBC BASIC needs none of this: it lives on the co-processor, out of reach of anything running here.

2.4 Aliases

`ALIAS` gives a name to a line.

```
ALIAS                   list them
ALIAS LL DIR A          define one
ALIAS LL                nothing after the name: remove it
```

Whatever you type after the alias is added to the end, so a definition takes arguments. Aliases are tried *last*, after every other rule, so one can never hide a real command: `ALIAS DIR ECHO no` is accepted and `DIR` still lists the directory. They live in memory — in a sideways bank, not in the 64 KB — so they survive a reset but not a power cycle, which is what `/STARTUP.BAT` is for.

2.5 The network

A URL is a file name. Anything the shell reads — `TYPE`, `LOAD`, `CP`, `RUN`, and the bare-word rule above — accepts `http://` or `https://` in place of a name, and the machine fetches it:

```
TYPE https://raw.githubusercontent.com/mlongval/bmc-k4510/master/README.md
CP http://example.org/game.prg game.prg
RUN http://example.org/game.prg
```

EhBASIC's `LOAD` and BBC BASIC's `LOAD` on the Tube take URLs the same way. The idea is the C64's Meatloaf cartridge, where a disk name could be an address on the internet; here it costs the ROM nothing, because the ROM never looks at a name — the host does the fetching. A typed line is 95 characters, and so is the longest URL.

A `tnfs://` URL is more than a file: it is a place. TNFS is the little UDP file protocol the FujiNet and Meatloaf servers speak, and the machine's current directory can be on one of them:

```
CD tnfs://fujinet.online/
DIR
CD CBM
DIR
CD -
```

`DIR` lists the server, a bare name loads from it (so `RUN`, and the bare-word rule, run programs straight off the internet), `CD ..` climbs, `CD -`

comes home. The prompt shows where you are. The server is read-only from here.

For programs that want a live connection there is the *N: device* at `$D900` (FujiNet's name for it): four channels, each a URL opened for reading and writing — `tcp://host:port` for a connection, `http://` for a page. `TELNET host port` is the demonstration, a `telnet.prg` in `/PRG`: what you type goes out, what arrives is drawn by JIM, the terminal (Chapter 4), so a BBS gets its ANSI colours and CP437 art and the cursor and function keys go out as VT sequences; F12 hangs up (Escape belongs to the far end). It turns the screen black while it is connected, because that is what BBS art is drawn for, and puts your colours back when it hangs up. The register map is in Chapter 10 and `core/net.h`. On the Pi the network is the Ethernet port (DHCP); without a cable the device answers "not fitted" and URLs are simply absent names. The Pi has no TLS, so `https://` works on the desktop only, and its network wakes on first use rather than at boot.

So that there is no guessing, this is the whole list of what the machine speaks, and where:

Scheme	What	Desktop	Pi
`http://`	fetch a file; a name anywhere a name goes	yes	yes
`https://`	the same, over TLS	yes	no
`tnfs://`	a directory on a TNFS server, read-only	yes	yes
`tcp://`	a raw connection, through the N: device only	yes	yes

There is no FTP, on either machine, and no SSH. The Pi's network is untested at the time of this printing.

2.6 CAPSLOCK

`CAPSLOCK` toggles a caps-lock: while it is on, letters read from the keyboard come up uppercase, so a language whose keywords are uppercase — BBC BASIC, EhBASIC — needs no Shift held. Digits and symbols are untouched, so it is a caps lock and not a shift lock, and Shift gives you the *other* case while it is on, the way a caps lock has always worked. It is also suspended while a program is running, so a program

that wants lower case — `:q` in VI, say — gets it, and the lock comes back when the program does. `CAPSLOCK ON` and `CAPSLOCK OFF` set it explicitly. It works from inside a BASIC through the * escape: `*CAPSLOCK` in BBC BASIC or EhBASIC.

2.7 Scripts and the boot file

`EXEC name` runs a text file as shell commands, one per line. A line whose first character is `#` is a comment, and a blank line is ignored. At power-on the machine runs `/STARTUP.BAT` as such a script, if it exists — straight in, with no pause and nothing to catch. It is the place to put your aliases; `/SYSTEM/STARTUP.SAMPLE` is one to copy from. If one ever stops the machine booting, page 16 is the way out.

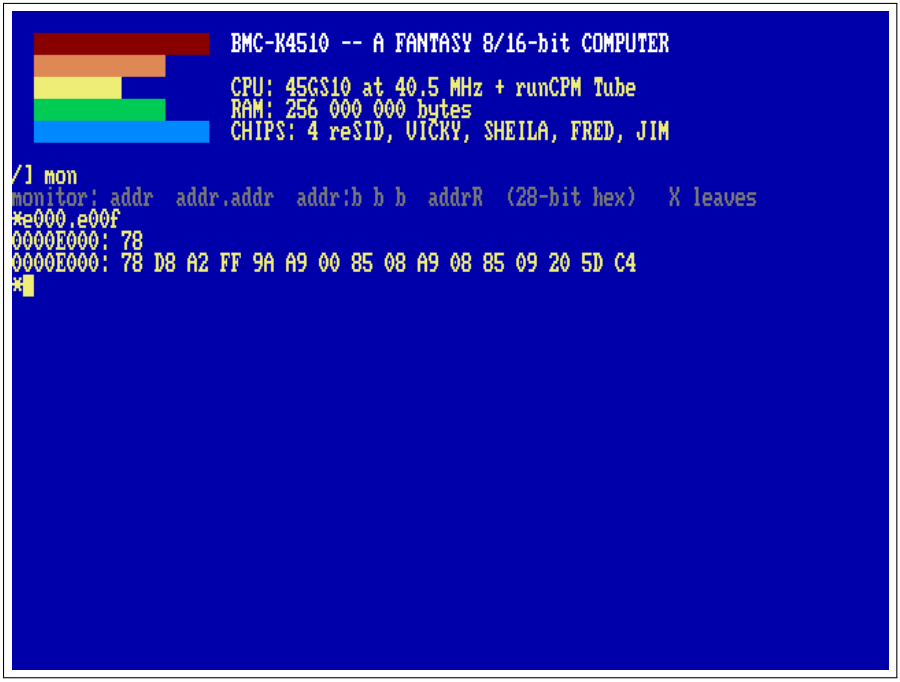
2.8 The monitor

`MON` (old-timers may type `WOZ`) opens the machine monitor at a * prompt; its grammar is Wozmon's, with 28-bit addresses:

```
*E000.E00F          examine a range
*0300:A9 41 60      store bytes
*0300R              run from $0300
*X                  leave
```

2.9 Housekeeping

`INFO` is the machine's self-description; `TIME` the clock; `MODE` and `COLOR` set the text screen — `CLS` clears it and `CLG` clears the bitmap over it, whoever drew it — both reach a BASIC through the * escape, which is how you tidy up after a demo that left its picture behind; `FILL` and `COPY` work on memory; `HUSH` silences all four SIDs at once; `LS` is `DIR` for fingers that have used Unix too long. `LOGO` clears the screen and prints the startup banner again — the colour bars and the machine's own description — which is a tidy way to end a session or start a screenshot. `RESET` restarts the machine from the shell, the same cold start the reset chord performs.



The monitor examining the ROM. `MON E000.E00F` runs one line without entering the prompt.

`BENCH` measures the machine: how many frames a second this host manages at the clock in force, then each clock of the menu's ladder for two seconds with a note sounding on SID 0, reporting frames and gaps in the sound for each, and the clock is put back. About 25 seconds; the report goes to `/SYSTEM/BENCH-NN.TXT`. A clock is right for this host when it holds 60 frames a second with no gaps — the machine's *optimal MHz* on that host. A recent desktop holds a hundred or more; a Pi 3B+ holds fifteen; and that figure, not the setting, is the one to quote when a program seems slow.

Several commands answer to more than one name, so that habits from another machine mostly work: `CD` is also `CHDIR`, `RM` is `DEL` and `ERASE`, `REN` is `RENAME` and `MV`, `DIR` is `LS`, `COLOR` is `COLOUR`, `XD` is `HEX`, `BBCBASIC` is `BBC`, and `CAPSLOCK` is `CAPS`. `CP` and `COPY` are *not* the same command: `CP` copies a file, `COPY` copies memory.

`MODE` on its own says where you are; `MODE n` moves. The second number is the margin: 1 (the default) keeps a one-cell gap at the top and the left, 0 uses every cell.

MODE	pixels	text	
0	640×480	80×60	the whole glass
1	640×240	80×30	the machine starts here
2	320×240	40×30	

The F7 menu, under Video, has the same two knobs: *Resolution* shows what the machine is actually in — it follows a `MODE` you type — and choosing another asks the ROM to perform it, because the console's geometry belongs to the ROM and not to the host. *Left/top margin* is the second parameter.

The menu offers the same three, and what it saves is never smaller than 320×240. VICKY can draw smaller fields than these, but they are for a program that wants the pixels, not for a shell; a program asks for them through the video registers (Chapter 10).

The picture is always painted into 640×480: a 240-line mode has each line drawn twice, and a 320-wide one has its pixels doubled sideways. Raster lines and SHEILA's display list count the glass, 0–479, not the mode — which matters the moment a program asks VICKY for a 200-line field, where the first line of the picture is raster line 40.

2.10 When STARTUP.BAT is the problem

`/STARTUP.BAT` runs at power-on, and a bad one runs at every power-on. There is no moment to catch — the machine boots straight into it — so the way out is not a key held at the right instant but a switch that stays where you put it: the F7 menu, under Shell: *Run STARTUP.BAT*. That setting is the host's, kept in `k4510.cfg`, so it survives a power cycle and no wedged program in the machine can take it away from you. Boot clean, fix the file, switch it back on.

On the desktop there is a second way, for one boot only:

```
./k4510 --no-startup.bat
```

(`--no-startup` does the same, as does `K4510_NO_STARTUP=1` in the environment). It skips `/STARTUP.BAT` for that run and touches nothing: the F7

setting and `k4510.cfg` are left exactly as they were. That is the one to reach for in a script, or when a startup file has wedged the machine and you want a single clean boot to go and fix it.

2.11 When something goes wrong

Type `DUMP ON`, make it go wrong again, then type `BUG` and answer its questions: the report it writes is the issue. Appendix B is the whole of it, on two pages.

Chapter 3

EhBASIC

The machine speaks EhBASIC 2.22 — Lee Davison's Enhanced BASIC — with this machine's additions: graphics statements that ride the blitter, sound on four SIDs, floating point on the MATH unit, and an escape hatch to the shell.

```
RUN EHBASIC
```

It comes up ready: the banner, `47103 Bytes free` — more than twenty-one C64s more than a C64 — and a `Ready` prompt — the traditional `Memory size ?` question is gone; the machine knows.

3.1 Ten minutes of it

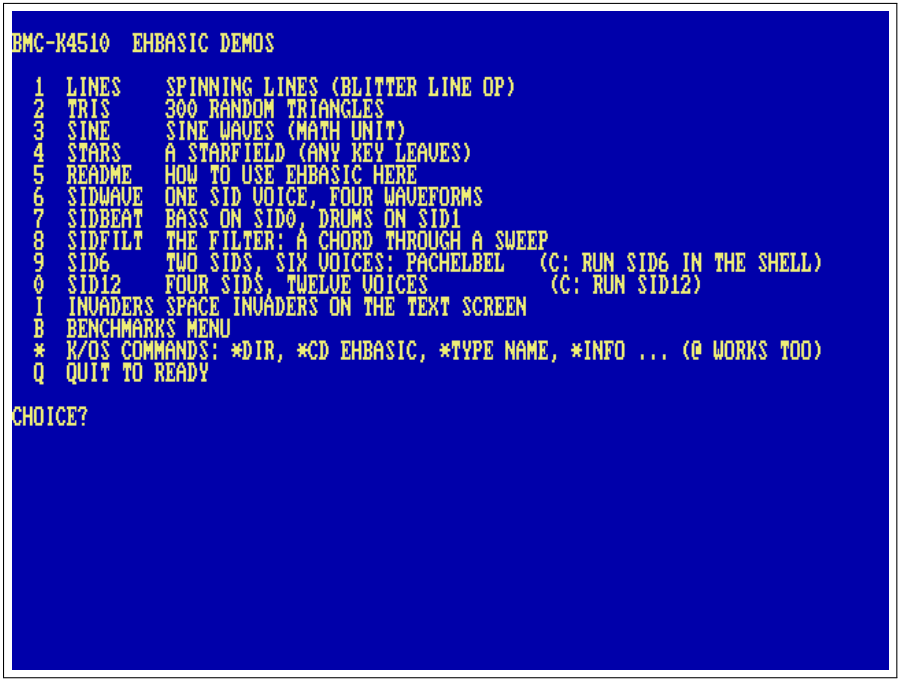
```
RUN "DEMOS.BAS"
```

The menu chains: `RUN "name"` (or `LOAD` inside a running program) loads another BASIC program and runs it — that is how one BASIC program hands over to another, exactly as on a C64.

3.2 The machine's own statements

```
GRAPHICS 2          640x480 bitmap over the text
PLOT X,Y,C          LINE X1,Y1,X2,Y2,C
TRI X1,Y1,X2,Y2,X3,Y3,C
PALETTE I,R,G,B     GCLS            GRAPHICS 0
```

Colours 0–15 belong to the text screen; demos use 16 and up. `GRAPHICS 0` puts the text mode and its palette back, whatever the program changed.



The demo menu. A key picks an entry; each demo returns here when it ends.

3.3 Hardware sprites

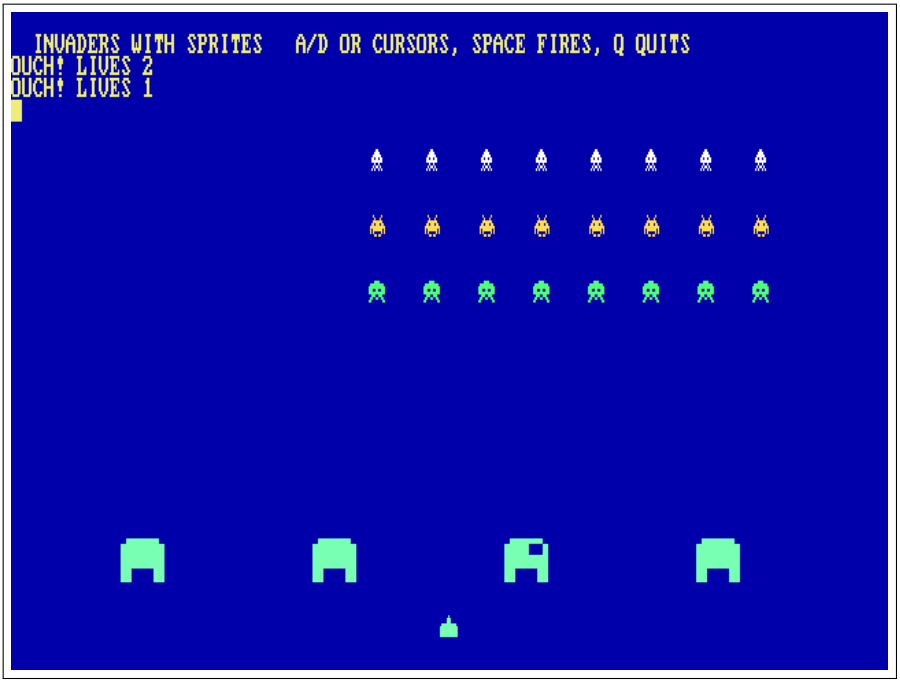
The 128 sprites VICKY carries are reachable from BASIC, which is unusual enough to be worth saying plainly: this is not a bitmap being redrawn, it is the video chip carrying the picture for you.

```
SPRDEF n,page,w,h,bpp   give sprite n a shape
SPRITE n,x,y            put it there and show it
SPROFF n                hide it
```

`SPRDEF` says where the pixels are and what shape they make: *page* is the 256-byte page the data starts at, which is how a 16-bit BASIC number reaches into 256 MB (`page*256` is the address); *w* and *h* are 8, 16, 32 or 64; *bpp* is 4 or 8. `SPRITE` positions the sprite and turns it on, `SPROFF` hides it. Colour 0 is transparent, and there is no per-line limit — all 128 may be on the same raster line.

Each statement re-points the chip at the attribute table and re-enables sprites, so a `GRAPHICS 0` or a shell `MODE` in between costs you nothing: the next sprite statement puts them back.

`INVADER2.BAS` in `/EHBASIC` is the demonstration — Space Invaders with the arcade shapes, poked 4 bits-per-pixel into $040000 through a bank register, four bases that erode as they are hit, and thirty-one sprites moving. `INVADERS.BAS` beside it is the same game on the text screen, which is a fair way to see what the sprites buy you.



`INVADER2.BAS`: twenty-four invaders, four bases and a cannon, every one of them a hardware sprite. The bases erode because their shape pages are poked, not redrawn.

3.4 The * escape

Any shell command works from BASIC with * in front — `*DIR`, `*CD EHBASIC`, `*MON`, `*DUMP a note` — and `*BYE` leaves BASIC for the shell. Escape or

Ctrl-C stops a running program (and silences the SIDs); the program's variables survive for PRINT-style post-mortems.

3.5 Editing the program in VI

*VI and *EDIT, with nothing after them, edit the program that is in memory:

```
10 PRINT "HELLO"
*VI
```

BASIC writes the program to a temp file, opens the editor on it, and reads it back when you leave — so what you type in the editor is what you LIST afterwards. Give either one a file name and nothing special happens: *VI notes.txt is the ordinary * escape, and the editor edits that file.

The trip out and back is not free. The editors load where the interpreter lives, so the shell command underneath is SWAP, which puts all 64 KB and the screen aside first and restores them afterwards. And because the program is written out and read back, *variables do not survive* — this is an immediate-mode thing, like LOAD. The temp file (EDITTMP.BAS) is left behind, which is occasionally a useful accident.

Why the first line is blank: SAVE starts its output with a newline, so the editor opens on an empty line 1 with the program below it. Harmless — BASIC ignores it on the way back in — but it is why the line count is one more than you expect.

Chapter 4

The Tube: BBC BASIC

The BBC Micro's most elegant idea was the Tube: a fast port through which a *second processor* — another CPU with its own memory — could take over the computation while the Beeb kept the keyboard, the screen and the discs. The BMC-K4510 has a Tube of its own at `$D800`, and the first thing fitted to it is Richard Russell's BBC BASIC, running on the host machine with a flat 256 MB of its own.

```
BBC
```

You get the > prompt of a BASIC with real power behind it, and it is *fast* — the co-processor is not cycle-matched to 1985:

```
HIMEM=PAGE+250*1024*1024
DIM space 200*1024*1024
```

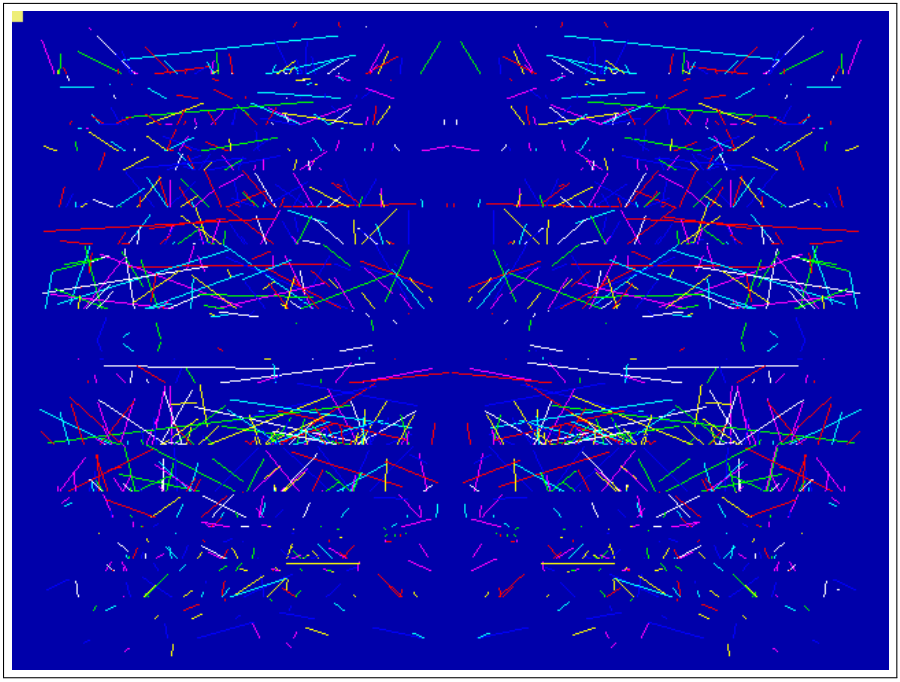
both just work. Type `*QUIT` to hand the console back to the shell.

4.1 Graphics and sound

The console cannot show what it cannot see, so the Tube speaks up: MODE, GCOL, PLOT, MOVE, DRAW, CIRCLE and the palette arrive at the machine as escape sequences, and the Tube's gatekeeper (the *Tube ULA*) executes them on the VICKY blitter. `SOUND` and `ENVELOPE`-less music go the same way, onto the machine's four-channel sound sequencer and out through the SIDs — the classic Beeb `SOUND chan,amp,pitch,dur`, queued per channel like the original.

```
LOAD "BBCBASIC/KALEID.BBC"
RUN
```

The `/BBCBASIC` directory carries a period programme selection: `KALEID`, `CIRCLES`, `ROSES`, `MOUNTAIN`, `BOUNCE`, `CLOCK` (a live analogue clock



`KALEID.BBC`: MODE 2 kaleidoscope, drawn by BBC BASIC on the Tube, painted by VICKY.

face) and `TUNE` — Frère Jacques as a three-voice round, the sound sequencer's party piece.

4.2 Sprites

VICKY has 128 hardware sprites, and BBC BASIC reaches them the way RISC OS reached its own: `VDU 23,27` selects and shapes, `PLOT &ED` places. There is no sprite file. A sprite is *captured* from the bitmap after BBC BASIC has drawn it with the words it already knows:

```
MODE 2
GCOL 0,1:CIRCLE FILL 40,40,30
MOVE 8,6:VDU 23,27,1,0,32,32|
CLG
PLOT &ED,600,500
```

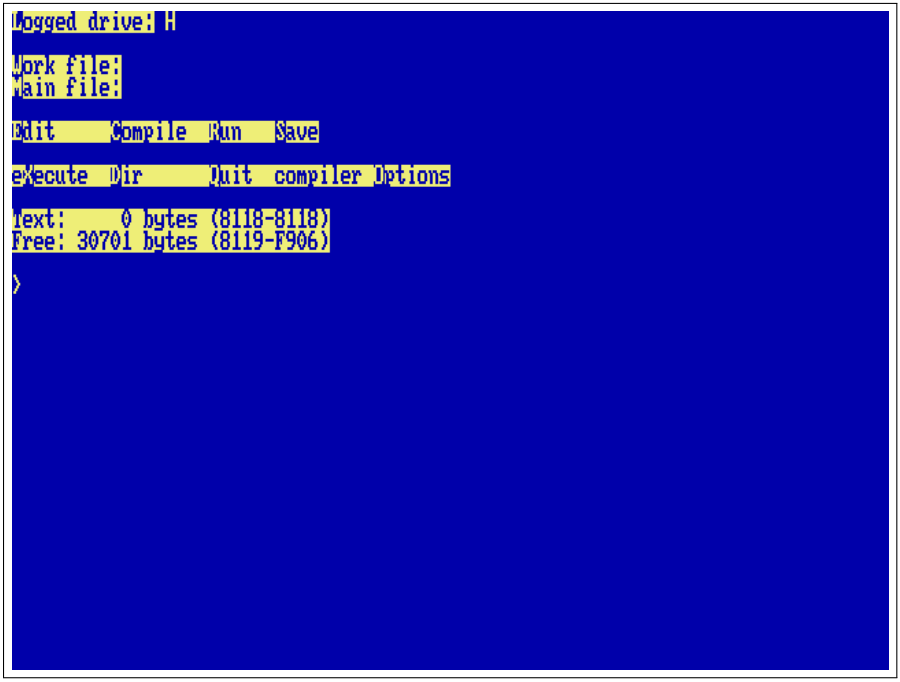
`VDU 23,27,0,n|`	select sprite n (0–127) for PLOT
`VDU 23,27,1,n,w,h|`	capture n, $w \times h$ pixels (8/16/32/64), bottom-left at the graphics cursor
`VDU 23,27,2,n|`	hide n
`VDU 23,27,3,n,f|`	flip: bit 0 horizontal, bit 1 vertical
`VDU 23,27,4,n,z|`	depth: drawn after layer z (default 1, over the bitmap)
`VDU 23,27,5,n,m|`	sprite n shows sprite m's picture
`PLOT &ED,x,y`	show the selected sprite, bottom-left at x, y

A placed sprite is a register write: moving sixty-four of them every frame costs the interpreter nothing but the PLOTs. `SPRITES.BBC`, `INVADERS.BBC` and `TUNNEL.BBC` in `/BBCBASIC` are the demonstrations (the last one moves nothing at all — it cycles the palette with `VDU 19`).

4.3 The terminal: JIM

The console you see when the Tube is running is not the ROM's own terminal but *JIM*, at `$DA00` — the Beeb's third I/O page, given a job here. JIM is a VT100 with the ANSI colours and the VT220 editing sequences (insert and delete character and line, erase character, scroll regions, origin mode, DEC line drawing, cursor-position and identity reports), built into the machine the way a real 8-bit computer got a serious terminal: as a card, not a program. It draws on the same text screen as the ROM console, inside the window the ROM gives it, so the two share one screen and one cursor. Anything that needs a terminal writes its byte stream to `$DA00` and reads its answers from `$DA02`; keys pushed through `$DA03` come back as the VT sequences the far end expects (arrows, Home/End, PgUp/PgDn, F1–F12, Delete as `$7F`). The register map is in `core/term.h`.

For CP/M this settles the question every program asks at install time: tell WordStar's `WSCHANGE`, Turbo Pascal's `TINST`, ZDE and the rest that the terminal is a *VT100* (or *ANSI*: the same sequences). Turbo Pascal 3



Turbo Pascal 3 (`H:` user 3) on JIM: the menu bar in reverse video, the compiler's own screen handling on a VT100 that is a chip.

on `H:` user 3 comes up with its menu already right; WordStar 4 on `E:` wants one pass of `WSCHANGE`. BBC BASIC's console edition speaks the same language natively, so `COLOUR`, `CLS` and `PRINT TAB(x,y)` land where they should. Bytes `$80`–`$FF` are drawn as CP437 glyphs, which is what BBS ANSI art is made of.

4.4 Star commands

A line starting with * goes to the machine: `*DIR`, `*CD`, any shell command — and BBC BASIC's own file words (`LOAD`, `SAVE`) read and write the machine's filesystem directly, because the co-processor lives inside `fs/` too. The Tube starts in the shell's current directory, so `CD BBCBASIC` then `BBC` lets `LOAD "KALEID.BBC"` work with no directory prefix; the machine's filesystem is the co-processor's whole world — `fs/` is shown as /, the

host tree above it hidden, and `*CD ..` stops at the root.

Edges, honestly: `POINT(` and `TINT` are not implemented; `GCOL` modes 1–4 draw plain; `VDU 5` text prints as text. On the Pi both co-processors run on core 3, the second processor.

Chapter 5

Forth

The machine's third language is native: no Tube, no co-processor, just 45GS10 machine code loaded at `$4000`. It is Tali Forth 2 — a public-domain, ANS-flavoured Forth written for the 65C02, which this CPU speaks as a strict superset.

```
FORTH
```

```
2 3 + . 5  ok
: cube dup dup * * ;  ok
7 cube . 343  ok
```

5.1 The machine at your fingertips

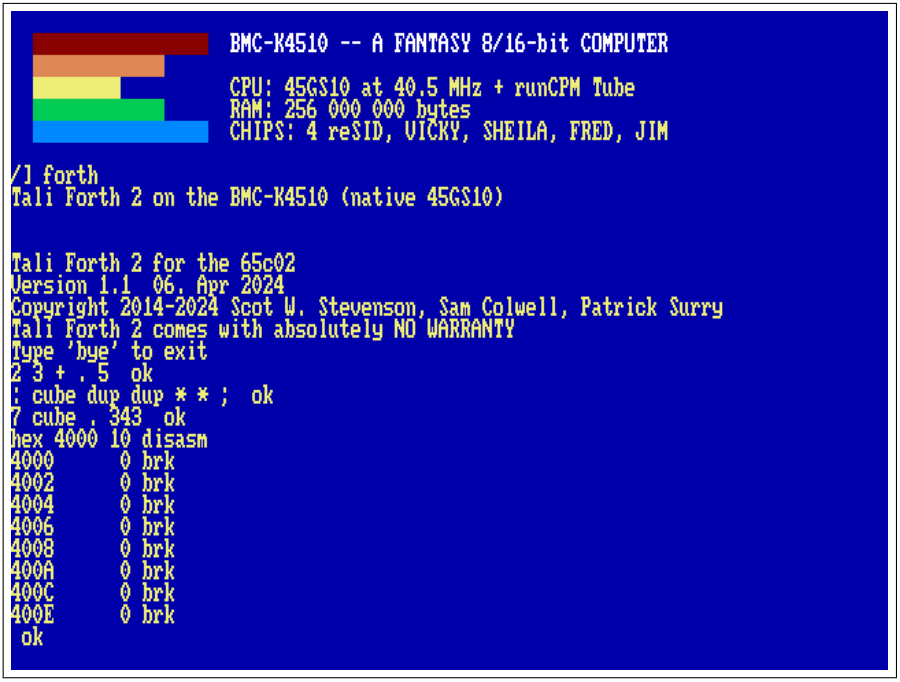
Forth's oldest habit is poking hardware, and this machine is one large, friendly memory map. `C@` and `C!` reach every register of Chapter 10 directly:

```
hex
D000 C@ .            read a VICKY register
D5E0 80 swap C!      HUSH, by hand: silence the sequencer
```

5.2 The assembler

Tali brings an interactive 65C02 assembler and a disassembler, kept in because on this machine they are the point:

```
4000 10 disasm       disassemble the Forth kernel itself
assembler-wordlist >order
here  2 lda.#  0 sta.z  drop
```



A Forth session: the banner, arithmetic, a new word, and the kernel disassembling itself.

`DISASM` is strictly 65C02: at a 45GS10-only opcode it prints a polite `?` and moves on.

5.3 Manners

The dictionary has about 31.7 KB free for your words. `BYE` returns to the shell with the screen and the session exactly as you left them.

> No file words yet: Forth source lives in `/FORTH` but must be typed. An `INCLUDED` that reads through the machine's storage device is on the list.

Chapter 6

CP/M: the Z80 Second Processor

In 1984 Acorn sold a Z80 Second Processor for the BBC Micro; its purpose was to run CP/M, the operating system that owned business computing before the IBM PC. It sat on the Tube. So does ours. `CPM` at the shell boots CP/M 2.2 on a Z80 co-processor (RunCPM, on the host), and the `A0>` prompt that defined a decade appears. The boot banner cheerfully measures the Z80 in *gigahertz* — read that number knowingly: it is the emulated Z80's speed *on whatever host you run the emulator on* (a laptop, in this book's screenshots), so it varies machine to machine. For scale, the real thing shipped at 4 MHz.

```
CPM
```

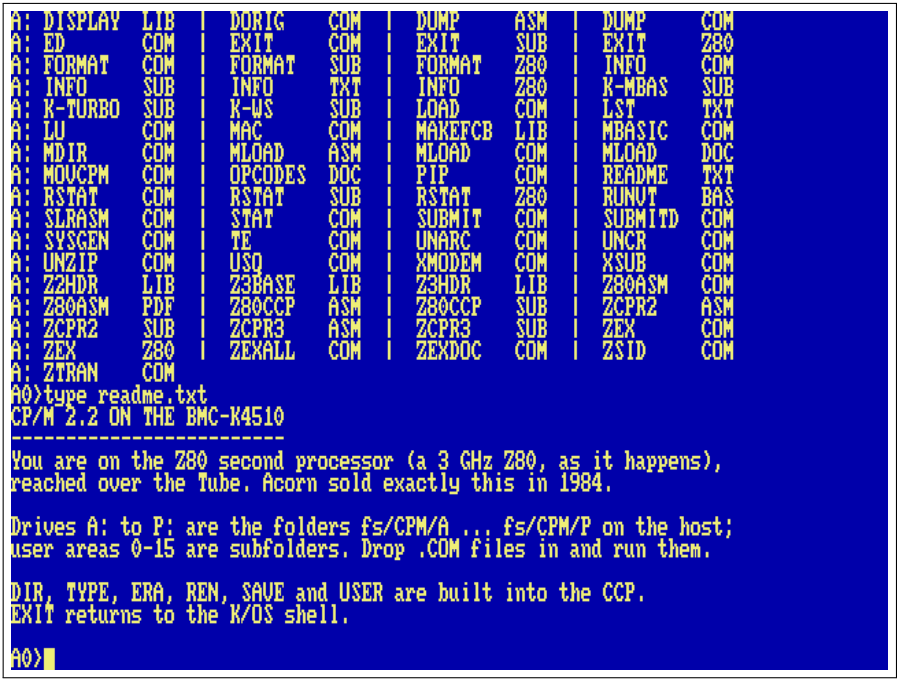
6.1 Drives are folders

Drives `A:` to `P:` are the host folders `fs/CPM/A` through `fs/CPM/P`; CP/M's user areas 0–15 are numbered subfolders. Drop any `.COM` file into `fs/CPM/A/0` and run it by name. `DIR`, `TYPE`, `ERA`, `REN`, `SAVE` and `USER` are built into the CCP; `EXIT` hands the console back to the K/OS shell.

Three of the drives are shortcuts rather than folders of their own: `K:` is the machine's own filesystem, so CP/M and K/OS can hand files to each other in both directions; `P:` is Turbo Pascal and `D:` is WordStar, each pointing straight at the user area the program lives in.

6.2 Starting a program from the shell

`CPM` on its own gives you the `A0>` prompt. `CPM command` runs a command at boot instead, through the CCP's `AUTOEXEC.TXT`. One line is all the CCP reads — but a name with no `.COM` to match runs the `.SUB` of that name, and that may be as long as you like. `/CPM/A/0/K-TURBO.SUB` is the pattern:



CP/M 2.2 over the Tube: the CCP's `DIR` and `TYPE`, on drive A.

```
P:
TURBO
EXIT
```

Ending in `EXIT` is what makes the round trip whole: quitting a CP/M program only drops you to the CCP, and the `EXIT` carries you the rest of the way back to the K/OS prompt. With that in place `ALIAS TURBO CPM K-TURBO` makes `TURBO` a word you can type at the shell; `/SYSTEM/STARTUP.SAMPLE` defines that one, `WS` and `MBASIC`.

Why not `USER` in a submit. The obvious script is `H:`, `USER 3`, `TURBO` — and it dies on the second line. The submit in progress is a file, `$$$.SUB`, and it lives on A: user 0; `USER 3` walks away from it and it cannot be read any more. That is CP/M's own behaviour and not RunCPM's doing. Pointing a drive at the user area, as `P:` and `D:` do,

keeps `USER` out of it.

6.3 Typing a CP/M program's name directly

The shell can be told to treat an unknown word as a CP/M program: F7 → Shell → *CP/M .COM by name*. With it on, `STAT` at the `/]` prompt starts CP/M, runs `STAT.COM` from A: and comes back.

It is off to begin with, and deliberately so — `D` typed for `DIR` should not start a Z80 program. It changes only the guess: a command naming `CPM` outright, an alias among them, works either way. There is nothing in a `.prg` to tell the two apart, incidentally — its header is a load address and a run address, and a `.COM` begins with Z80 code that would read as a perfectly plausible one — so the extension is what decides.

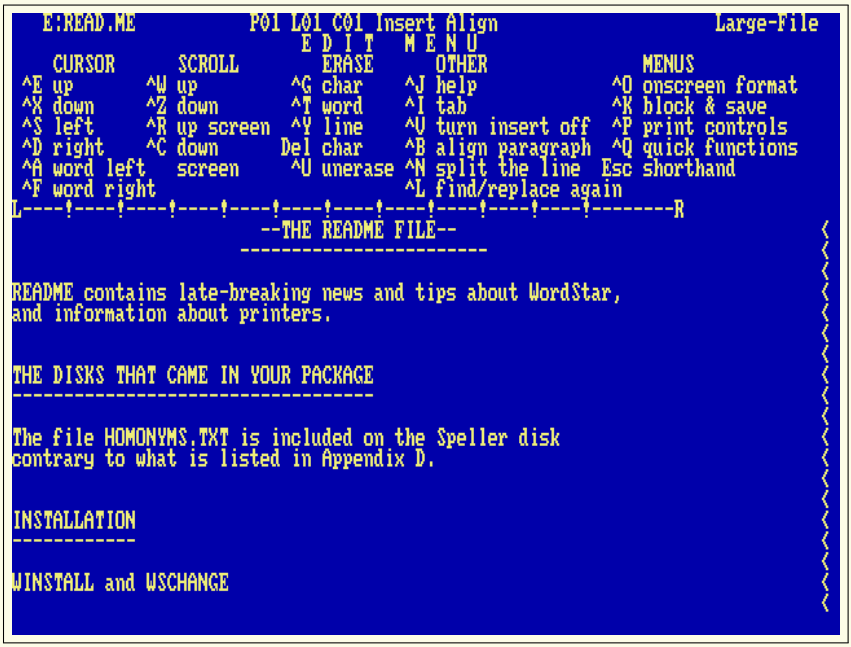
6.4 Software

The RunCPM project ships a complete system disk (`DISK/A0.zip` in its repository): Digital Research's own `ASM`, `MAC`, `DDT`, `ZSID`, `STAT`, `PIP` and `ED`, the `SUBMIT` batch system, Microsoft BASIC, a Z80 assembler with its manual, XMODEM, and the source code of the BDOS and CCP — buildable on the machine itself. One `unzip A0.zip -d fs/CPM/` installs the lot. Beyond that lies the whole surviving CP/M software world: WordStar, Turbo Pascal, dBase II, Zork, one archive away.

Edges, honestly: line-mode programs (assemblers, compilers, MBASIC, adventures) work today, and so does the full-screen software: the console is JIM, a VT100 in hardware (Chapter 4). WordStar 4 on `E:` user 3 comes installed for "ANSI Standard" at 79×29 — `WS` and you are editing; Turbo Pascal 3 on `H:` user 3 needs nothing either. A program that asks its terminal type at install time wants "VT100" or "ANSI".

The four arrow keys work in CP/M software: while the Tube is running CP/M the machine sends them down as the WordStar keys — Ctrl-E, X, S and D for up, down, left and right — which is what the pro-

grams of 1984 read, WordStar and Turbo Pascal (Chapter 7) among them. The rest of the navigation keys do not: Home, End, PgUp and PgDn are two-key ^Q sequences in WordStar, and there is no single key the machine could honestly send for them, so type those as the program's manual says.



WordStar 4 editing its own `READ.ME`: the edit menu, the ruler, the text — 79×29 on JIM.

Chapter 7

Pascal

The machine has two Pascals, and they share nothing but the language. Keep them apart in your head from the start:

	Turbo Pascal 3	**Mad Pascal**
What it is	a program of 1985	a cross-compiler of today
Where it runs	on the machine: the Z80, under CP/M	on your desktop
What it makes	a CP/M `.COM`, for the Z80	a K/OS `.prg`, for the 45GS10
What it can touch	CP/M's world only	every chip in the machine
How you get it	you supply it; it is Borland's	in the repository, free

Nothing carries over between them — not the source, not the toolchain, not the programs they produce. The first section is one, the second is the other.

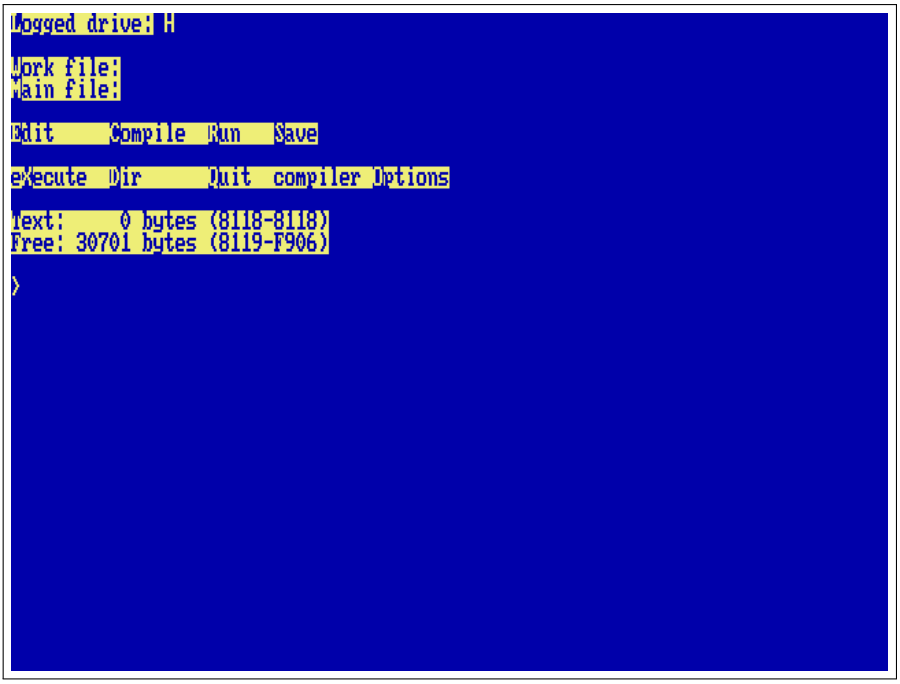
7.1 Turbo Pascal 3, on CP/M

Borland's Turbo Pascal 3 is a CP/M program, so it runs where CP/M runs: on the Tube's Z80 (Chapter 6), in its own world, exactly as it did on a Kaypro. You write, compile and run without leaving it, and what it produces is a CP/M program that runs under CP/M on the Z80 — it cannot see VICKY, the SIDs or anything else on the K4510 side of the Tube, any more than a Kaypro could.

It is not shipped, because it is Borland's. Put `TURBO.COM` and `TURBO.MSG` into `fs/CPM/H/3` — the folder that CP/M's drive `P:` is a shortcut to — and install it for a "VT100" or "ANSI" terminal, which is what JIM is. Then, from the K/OS prompt:

```
TURBO
```

`TURBO` is an alias that `/SYSTEM/STARTUP.SAMPLE` defines, for `CPM K-TURBO`: boot CP/M, go to `P:`, start Turbo, and — because the submit ends in `EXIT` — come all the way back to the K/OS prompt when you leave it. Chapter 6 explains that arrangement.



Turbo Pascal 3 starting on the Z80, its terminal drawn by JIM.

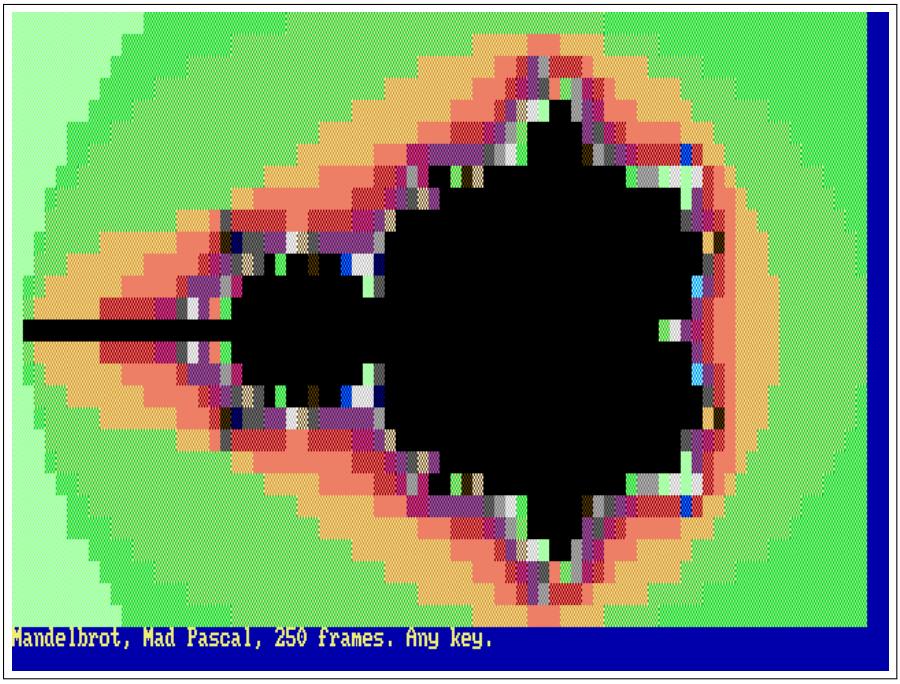
Inside, it is 1985. The editor is WordStar's: Ctrl-E, S, D and X move (the arrow keys are sent as those, so they work too; Home, End, PgUp and PgDn are not, and do not), and Ctrl-K then D leaves the editor for the menu. `R` runs the program in memory; `O` then `C` makes it compile to a `.COM` you can run from the CP/M prompt by name.

7.2 Mad Pascal, the cross-compiler

Mad Pascal is the other kind entirely — a cross-compiler, like the cc65 this machine's ROM is built with. You write on the desktop, `mp` compiles to 6502 assembly, MADS assembles it, and the result is a `.prg` in `fs/PRG`

that the shell's `RUN` loads like any other program. It is Tomasz Biela's compiler (MIT), a Turbo-Pascal-flavoured language with 8-, 16- and 32-bit integers, fixed and floating point, strings, records, pointers, units and inline assembly, made for the Atari and since taught the C64, the X16 and the Neo6502. The K4510 is its newest target.

```
PMANDEL
```



`PMANDEL`: the Mandelbrot set in text, sixteen colours through JIM, in 250 frames.

That is `demo/pas/pmandel.pas`: the Mandelbrot set, 78 by 28 cells in the machine's sixteen colours, in a little over four seconds at the desktop's 40.5 MHz — fixed point, 32-bit integers, forty lines of Pascal. `PSIEVE` is the classic sieve, ten passes timed by the frame counter; `HELLO` prints, shows the colours, and echoes what you typed after its name. All three are already compiled and in `fs/PRG`, so you can run them without installing anything.

Building

On the desktop, with FPC and a Mad-Pascal checkout (the installer patches its target table and rebuilds `mp`). It looks for the checkout in `~/Projects/neo6502_dev/Mad-Pascal` unless `MP_DIR` says otherwise:

```
make pascal-install    # once; MP_DIR=... to point elsewhere
make pascal            # demo/pas/*.pas -> fs/PRG/*.prg
```

A new program is a file in `demo/pas/`; `make` finds it. The compiled `.prg` files are committed, so a machine without the toolchain still runs them, and the test suite (`test/pastest.sh`) runs `HELLO` and `PSIEVE` through the ROM.

The target

Everything a Pascal program needs from the machine is in `pascal/` of the repository, laid out as it lives inside a Mad-Pascal checkout: the runtime base (`base/rtl6502_k4510.asm` and `base/k4510/`), where `@putchar` writes to JIM, the terminal (Chapter 4); the SYSTEM and CRT units' machine halves (`lib/*_k4510.inc`); and a `k4510` unit. The consequences for the programmer:

- `Write` and `WriteLn` go through JIM, so the CRT unit is the real thing: `GotoXY`, `TextColor` and `TextBackground` (the palette's constants: `BLUE`, `YELLOW`, `LIGHT_GREEN`...), `ClrScr`, `ClrEol`, `InsLine`/`DelLine`, `WhereX`/`WhereY`, `CursorOn`/`CursorOff`, `ReadKey` and `KeyPressed` on the keyboard device, `Delay` and `Pause` on the frame counter, `Sound` on SID 0. `TextMode(0)` puts the ROM's screen back.
- `ParamCount` and `ParamStr` come from the shell line (`RUN HELLO one two`, or just `HELLO one two`); `ParamStr(0)` is the whole tail.
- `uses k4510` gives every chip as a typed variable at its address — `VICKY[reg]`, `SIDREG[]`, `DMA_SRC`, `FS_CMD`, `SYS_FRAMES`, `MATH_F[]`, `TERM_CX` and the rest — plus `FarPeek`/`FarPoke` into the 256 MB through the 45GS02's flat addressing, `DmaCopy`/`DmaFill`, `Shell('DIR')`, `LoadFile`/`SaveFile` through the ROM, `WaitVBlank`, and `TermWrite` for raw escape sequences.

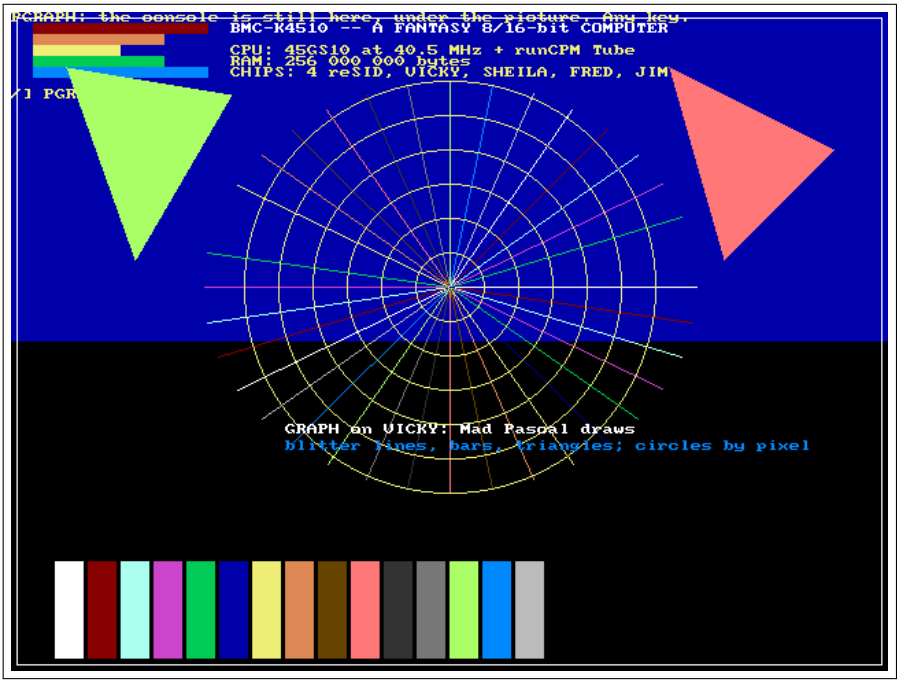
- Memory: code from $0800, the program's own; zero page $22--$3F for the compiler's registers and $64--$A3 for its expression stack (the ROM keeps $02--$21 and $F0--$F9); $0300 the string buffer. Programs return to the shell with the ROM's state intact.

Floating point on the MATH unit

`single` (IEEE-754, 32-bit) does not run in software here. The runtime's add, subtract, multiply, divide, compare, `Trunc`, `Round` and `Frac` are the MATH unit at $D700 (Chapter 10): each is a few register moves and one write to FOP, and the unit answers in the next cycle. `PFLOAT` times five thousand rounds of multiply, divide, add and subtract: 5 frames on the unit, 24 with the software library (assemble with `mads -d:SOFTFLOAT=1` to get it back for comparison). The transcendentals are in the `k4510` unit — `MathSqrt`, `MathSin`, `MathCos`, `MathTan`, `MathAtan`, `MathAtan2`, `MathExp`, `MathLn`, `MathPow`, `MathFloor` — one register write each. SYSTEM's own `Sqrt`/`Sin`/`Cos`/`ArcTan`/`Exp`/`Ln` on `single` run on the unit too: the installer patches `system.pas` under `{$ifdef k4510}`.

Graphics on VICKY

`uses graph` draws on VICKY's layer 1: a 640×480 8-bit bitmap at $200000 — the one BBC BASIC's ULA uses too — over the console, which stays visible wherever the picture is transparent (colour 0), as in a BBC MODE. `InitGraph(0)` switches the video to 480 lines and clears the bitmap; `CloseGraph` puts the console's mode back. `PutPixel` and `GetPixel` go through the 45GS02's flat addressing (the MEGA65 multiplier does the row arithmetic); `Line`, `LineTo` and the K4510 additions `DrawLine`, `FillBar` and `FillTriangle` are the blitter's LINE, FILL and TRIANGLE operations — one register write each; `Circle`, `Ellipse`, `Rectangle` and the rest of Mad Pascal's generic GRAPH build on those; `OutTextXY` writes with the console's font. `PGRAPH` is the demonstration.



PGRAPH: bars, a star and triangles by the blitter, circles by pixel, text from the ROM's font — and the console's own line still on top.

Chapter 8

The Editors

Two of them, because they answer different questions. `EDIT` is the one to reach for when a file wants a line changed and you want to be out again in ten seconds. `VI` is the one for a file too big to think about, and it is modal, so it expects you to have met `vi` before.

Both draw through JIM, the VT100 in hardware (Chapter 4), and both take their keys raw from the ROM — an arrow arrives as one byte rather than an escape sequence to unpick. Neither needs the Tube, so both run on the Pi as well as the desktop.

8.1 EDIT

`EDIT name` opens a file, or starts an empty one if there is no such file yet. The whole text sits in memory below the program, so it holds about 22 KB — ample for a `STARTUP.BAT`, a `.SUB`, a BASIC listing or a letter. There are no modes: what you type goes in, and everything else is a control key.

Key	Does
arrows	move by a character or a line
Home, End	start and end of the line
PgUp, PgDn	a screenful
Enter	split the line at the cursor
Backspace	rub out to the left; at the start of a line, join it to the one above
Delete	rub out under the cursor
Ctrl-O *or* Ctrl-S	save
Ctrl-X	leave — twice on a changed file, to throw the changes away

The bar along the bottom carries the name, a * while there are unsaved changes, where the cursor is, and those last two keys, so there is nothing to remember. Long lines slide sideways rather than wrap.

8.2 VI

`VI name` is modal in the old way. In *normal* mode the keys are commands; a handful of them put you into *insert* mode, where what you type goes into the file; and Escape brings you back. The `:` line at the bottom is the third place, for the ex commands.

Modes

Key	Enters insert…
`i`	before the cursor
`a`	after the cursor
`I`	at the start of the line
`A`	at the end of the line
`o`	on a new line below
`O`	on a new line above
`s`	replacing the character under the cursor
`S`	replacing the whole line
`C`	replacing the rest of the line
`c`*motion*	replacing what the motion covers; `cc` the whole line
Esc	…and leaves it, back to normal mode
`:`	opens the command line (from normal mode)

Moving

A count goes in front of nearly anything: `5j` is five lines down, `10G` is line ten.

Key	Moves
`h j k l`	left, down, up, right — the arrows do the same
`w`	to the next word
`b`	back a word
`e`	to the end of the word
`0`	to the start of the line
`^`	to the first non-blank on the line
`$`	to the end of the line
`gg`	to the first line
`G`	to the last line, or to line *n* with a count
PgUp, PgDn	a screenful
/*text*	to the next *text* — a plain substring, not a pattern
?*text*	to the previous one
`n`	the same search again; `N` the other way

Changing

Key	Does
`x`	delete the character under the cursor
`X`	delete the one to its left
`d`*motion*	delete what the motion covers: `dw` a word, `d$` to the end
`dd`	delete the line
`D`	delete to the end of the line
`y`*motion*, `yy`	copy (yank) the same, without deleting
`p`	put what was deleted or yanked, after the cursor
`P`	put it before
`r`*c*	replace one character with *c*
`~`	flip the case of one character
`J`	join the next line onto this one
`u`	undo — unlimited
Ctrl-R	redo — also unlimited

Operators take any motion — `d2w` deletes two words, `y$` yanks to the end of the line — or double themselves to work on whole lines. A charwise operator **clamps to the line**: `dw` at the end of a line stops there

instead of eating into the next one. That is deliberate, and so is the plain-substring search: `/foo` finds `foo`, and `.` `*` `[` mean themselves.

The command line

Command	Does
`:w`	save
`:w` *name*	save under another name
`:q`	leave — refuses on a changed file
`:q!`	leave anyway, changes lost
`:wq`, `:x`	save and leave
`:s/`*old*`/`*new*`/`	replace the first *old* on this line (`/g`: every one)
`:%s/`*old*`/`*new*`/g`	the same on every line of the file
`:map` *keys result*	define a key sequence, in normal mode
`:imap` *keys result*	the same, in insert mode

Making `jk` leave insert mode

The usual mapping is the one most people want first, and it is typed at the `:` line exactly like this, with the Escape spelt out in angle brackets:

```
:imap jk <Esc>
```

From then on, typing `j` then `k` in insert mode leaves it, and the two letters never reach the file. A lone `j` still does: the editor holds it for half a second waiting for the `k`, and hands it over when nothing comes. `<CR>` is the other spelt-out key; everything else in the result is literal.

To have it every time, put the same line, without the colon, in `/SYSTEM/VI.RC`: `VI` runs that file once the text is loaded, one ex command per line, a line starting with `"` being a comment. `/SYSTEM/VI.SAMPLE` is one to copy, the way `STARTUP.SAMPLE` is, and the file is yours — it is not in the repository.

Where the file lives. Nothing of it is in the 64 KB. Every line is a 256-byte slot in far memory — a length and up to 255 characters — and only the line under the cursor is brought down, fetched when the cursor arrives and written back when it leaves. So the ceiling is

far memory rather than the address space: 32000 lines, and LOAD and SAVE go straight to and from a flat copy out there without the file ever passing through the CPU's view. Undo is a journal in the same place, which is why it is unlimited: one level would have cost the same as all of them.
A 1.28 MB file of 20000 lines — twenty times the whole address space — opens, edits at the far end and saves back byte for byte.

8.3 Editing from inside a BASIC

There are two cases, and they look alike, so here is the rule.

To edit the program you are writing, type *EDIT or *VI with nothing after it. BASIC saves the program to a temporary file, runs the editor on it, and loads it back when you leave — so what you type in the editor is what you LIST afterwards. Variables do not survive the round trip, exactly as with LOAD. Section 3.5 has the details.

To edit any other file, put SWAP in front:

```
*SWAP EDIT NOTES.TXT
```

A program loaded from the shell lands on top of whatever is in memory, so a plain *EDIT NOTES.TXT would run the editor over the program you were writing. SWAP (Chapter 2) puts the machine away first — program, variables, screen and all — runs the editor on a clean one, and gives yours back when the editor exits.

So: no file name, no SWAP needed, it is done for you; a file name, SWAP first.

Part II

Programmer’s Guide

Chapter 9

Memory

9.1 The 64 KB view and the 256 MB behind it

The 45GS10's program counter is 16 bits: code executes inside a 64 KB window. The 256 MB of physical memory is reached three ways:

- **Flat pointers**: `LDA [ptr],Z` and the Q forms read and write any byte of the 256 MB directly. Data never needs banking.
- **Bank registers** (`$D600`, one per 8 KB block): write a 28-bit base and that block of the window shows that memory. Bytes 0–2 set the base; byte 3 switches (bit 7 = off).
- **DMA** (`$D200`): copy, fill, swap, line, triangle — instant, physical addresses.

9.2 The CPU view, unmapped

`$0000-$03FF`	system	zero page, stack, vectors, low system state
`$0800-$CFFF`	user	50 KB for programs, ROM-free at launch
`$A000-$BFFF`	ROM	*the sideways window; RAM underneath*
`$C000-$CFFF`	ROM	*RAM underneath, bankable*
`$D000-$DFFF`	I/O	always I/O, whatever is banked
`$E000-$FEFF`	ROM	*RAM underneath, bankable*
`$FF00-$FFFF`	stub	always ROM: system-call stub and vectors

9.3 RAM under the ROM

`rom_out()` (or three pokes to the bank registers) banks blocks 5–7 onto the RAM beneath the ROM: a program then owns `$0800-$CFFF` + `$E000-$FEFF` = 57 KB. Every system call still works — calls go through

the $FF00 stub page, which banks the ROM in and out around them. RUN romout demonstrates the whole mechanism on screen.

9.4 Sideways ROM

The operating system is bigger than its half of the 64 KB — so, like the BBC Micro before it, the machine grew *sideways*. The ROM file is a 24 KB base image plus appended 8 KB banks; cold commands live in the $A000–$BFFF window (bank 0 is the base image itself, INFO and TIME already run from bank 1), and the shell pages the right bank in around each call. System calls work throughout: every call already passes through the always-ROM stub page at $FF00, which banks the ROM view in and out. Up to sixteen banks — 128 KB of operating system — fit in the scheme.

The calls go one way, and this is the rule to hold on to if you write code of your own for a bank: code in a bank may call resident code as freely as it likes, but resident code must never call into a bank. At that address a different bank holds something else entirely, and the dispatch is the only thing that knows which bank is in. A routine that everything uses belongs in the base image, whatever it costs there.

9.5 RAM under the I/O page

A MAP of block 6 hides $D000–$DFFF and exposes RAM: with the ROM also banked away a program owns a contiguous field from $0800 to $FEFF — 61.75 KB of the 64. The trick that makes it safe is that MAP is an *instruction*, not a register: the program that hid the I/O can always ask for it back, so there is no way to lock yourself out. (The far-call gate is unreachable while block 6 is mapped; unmap before far calls.)

9.6 Programs bigger than the window

The K4SG executable format loads segments to any physical address and the far-call gate ($DF00) calls between them: a plain JSR into slot n banks descriptor n's code in, and the callee's RTS banks it back out. RUN segdemo shows two overlays sharing one address.

Chapter 10

The I/O Page

One page, `$D000`–`$DFFF`, always visible. The authoritative register-level reference is the machine's own headers, and Appendix A is generated from them at build time — the way the Neo6502 handbook generates its keyword reference from the firmware source. This chapter is the map of what lives where:

`$D000`	VICKY	video: layers, palette, blitter, sprites, SHEILA
`$D100`	input	keyboard FIFO, status, peek, break-pending
`$D200`	DMA	copy / fill / swap / line / triangle
`$D300`	storage	the host filesystem: open, read, dir, chdir…
`$D400`	SID 0-3	four chips, 32 bytes apart
`$D500`	system	clock, RTC, frame counter, DUMP, SID crystal
`$D600`	banks	the eight bank registers, masks at `$D620`
`$D700`	MATH	IEEE floats, transcendentals, math lists, mul/div
`$D800`	Tube	the co-processor: status, byte in, byte out, start/stop
`$D900`	N:	the network: four channels, tcp://, http(s):// and tnfs:// (see `core/net.h`)
`$DA00`	JIM	the terminal: a VT100/ANSI in hardware, drawing on the console's screen (see `core/term.h`)
`$DF00`	far gate	call slots, descriptor table pointer, depth

Chapter A

The Registers

Everything the machine's devices do, they do through the I/O page. This appendix is the register-level reference, and it is not written by hand: `doc/guide/mkregs.py` lifts it out of the machine's own headers at build time, so what you read here is what the emulator was compiled against. When a device gains a register, this appendix gains it in the next build — which is the only way a reference of this kind stays true.

Nothing in it is reworded: every word below is a word from a header. What the generator decides is only which column a word belongs in — the address on the left, the register's name after it, the comment's own sentence following. Where a comment's own spacing was doing the work of a table, that table is kept exactly as it was written, in monospace, because there it is the spacing that carries the meaning.

The voice is still the voice of working code: terse, occasionally opinionated, and using the machine's own shorthand — `LE` for little-endian, `R:` and `W:` for a register that reads and writes differently, `$` for hex. Addresses are given as an offset inside the device's block where the block's base is obvious, and in full where it is not.

Chapter 10 is the map of which device lives where; this is what is inside each one.

A.1 The I/O page

Generated from `core/io.h`.

Bank registers and the far gate

BANK registers: `$D600` + 4n, n = 0..7, one per 8 KB block of the CPU view.

`bytes 0-2`	physical base bits 0-23 (little-endian); they only set the base
`byte 3`	bits 24-27 of the base, and bit 7 = OFF. Writing byte 3 switches the block: bit 7 clear = on, set = off. (So STQ works, and a

byte-wise save/restore never switches a block on by accident.) Reads give the base; byte 3 reads $FF while the block is off.

$D620 read: bit n = block n banked

$D621 read: MAP mask (bit n = MAPped)

A banked block resolves phys = base + (cpu & $1FFF). MAP rewrites all eight blocks, so the ROM's "MAP off" at program exit clears the banks too. The I/O page $D000-$DFFF and the stub page $FF00-$FFFF stay visible whatever is banked, so banking blocks 6 and 7 reveals the RAM under the ROM only. FAR gate: JSR $DF00 + 4n calls descriptor n of the table at FARTAB.

$DF00-$DF7F 32 call slots

$DFF0 return gate (RTS lands here)

$DF80-$DF83 **FARTAB** 28-bit pointer to the descriptor table (read/write)

$DF84 read: nesting depth

$DF85 read: last error (1 overflow, 2 underflow, 3 bad slot); write clears

descriptor, 8 bytes: base[4] block flags entry[2]; flags bit0 leave banked on return, bit1 do not bank (long jump to resident code). A/X/Y/Z pass through both ways: cc65 __fastcall__ works across it.

The MATH unit

MATH unit. Results are ready the cycle after the write that triggers them.

$D700-$D71F **F0..F7** eight IEEE-754 single registers, little-endian, read/write

$D720 **FOP** write = execute; low 5 bits op, see below

$D721 **FARG** (dst « 4) | src, register numbers 0..7, write before FOP

$D722 **FFLAGS** from the last op: bit0 result zero, bit1 negative, bit2 NaN/inf

$D724-$D727 **FI** int32 for ITOF / FTOI

```
ops: 0 MOV Fd=Fs  1 ADD  2 SUB  3 MUL  4 DIV   (Fd = Fd op Fs)
     5 SQRT 6 SIN 7 COS 8 TAN 9 ATAN 10 EXP 11 LOG 14 ABS 15 NEG 16 FLOOR 17 ROUND  (Fd =
         f(Fs))
     10 ATAN2 (Fd = atan2(Fd,Fs))  13 POW (Fd = Fd^Fs)  21 FMOD (Fd = fmod(Fd,Fs))
     18 CMP   flags from Fd - Fs, registers unchanged
     19 ITOF  Fd = (float)FI        20 FTOI  FI = (int32)Fs, truncated
```

Math list – a program for the unit in RAM, run with one write:

$D728-$D72B **MLPTR** 28-bit pointer to the list

$D72C **MLRUN** write = run from MLPTR until END or a STOP fires

$D72D	**MLSTAT** 0 reached END, 1 a STOP fired, $FF runaway (65536 ops)
$D72E,$D72F	**MLCNT** 16-bit counter for DJNZ, read/write
	list ops, 2 bytes each (op, arg) unless noted; ops 0..21 are the FOP ops with arg = (dst«4)|src, and in addition:
$80	**END**
$81	**STOPNEG** stop if last flags negative
$82	**STOPPOS** if not negative
$83	**STOPZERO**
$84	**STOPNZ**
$85	**JUMP** arg (signed, in ops from the next op)
$86	**DJNZ** arg MLCNT–, jump if nonzero
$87	**STOPFIGE** arg stop if FI >= arg (unsigned byte compare on the low byte, FI clamped)
$88	**LDF** arg=dst«4, then 4 bytes: IEEE single immediate into Fdst (6-byte op)
$89	**LDI**, then 4 bytes: int32 immediate into FI (6-byte op)
$8A	**LDMS** arg=dst«4, then 4 bytes: 28-bit address of a Microsoft-format float (exponent excess-128, mantissa1 with sign in bit 7, mantissa2, mantissa3 – EhBASIC's packed variable format); converted into Fdst (6-byte op)
	MEGA65-compatible integer unit (same addresses as the MEGA65):
$D770-$D773	**MULTINA**
$D774-$D777	**MULTINB** (unsigned 32-bit, LE)
$D778-$D77F	**MULTOUT** = A * B, 64-bit
$D76C-$D76F	**DIVOUT** integer part of A / B
$D768-$D76B	fractional part (32.32)
	recomputed on every write to an input byte; B = 0 gives all-ones.

System registers, and the SIDs

SYS registers (read-only unless noted):

$00,01	CPU clock, kHz, LE (40500)
$02,03	physical RAM, MB, LE (256)
$04	read: latch the host clock into $05–$0C and return 0
$05	sec

`$06`	min
`$07`	hour
`$08`	day
`$09`	month
`$0A,0B`	year LE
`$0C`	weekday (0=Sun)
`$0D,0E,0F`	frames since reset, 24-bit LE (vblank count)
`$10–$1F`	version string, NUL-terminated
`$20`	ROM base page (e.g. `$A0` for a 24 KB ROM)
`$F0`	write: DUMP – the host writes dumps/dump-NNN.txt (machine state, screen, PC history, keys, the shell log); read: the number of the last dump
`$F1`	write: append a byte to the shell log (the ROM logs command lines and DUMP notes)
`$F2`	write 1/0: automatic dump every 900 frames (15 s) on/off (on at reset on the desktop, off on the Pi; DUMP or DUMP ON also arms the per-instruction PC recorder); read: the setting
`$23`	read: the CPU clock setting in force (0 = 40.5 MHz, 1 = 30, 2 = 20, 3 = 15, 4 = 10); write: ask for one – the host applies it next frame (BENCH sweeps them)
`$24,25`	audio gaps LE: callbacks that found the ring empty since last cleared; any write clears
`$F3`	SID clock: 0 = 1 MHz (default), 1 = PAL C64 (985248 Hz), 2 = NTSC (1022730 Hz); read/write

SID registers `$00–$18` read back the last value written (a shadow; real SIDs are write-only there). `$19–$1C` come from reSID as on the chip.

The keyboard

Keyboard: a FIFO of key-down events. Printable keys arrive as ASCII (`$20–$7E`, already shifted/dead-keyed by the host layout on the desktop); control keys as ASCII controls; everything else as `$80+` codes.

The Tube

The Tube (`$D800`): Acorn's answer, refitted. The HOST runs Richard Russell's BBC BASIC interpreter (the vendored BBCTTY console edition, tube/bbcbasic)

on a pty; the machine talks to it byte-wise:

`$D800` *R* **status** bit0 alive, bit7 a byte waits in `$D801`

`$D801` *R* next byte from the co-processor (pops)

`$D802` *W* a byte to the co-processor (its keyboard)

`$D803` *W* 1 start (spawn), 2 stop (kill)

The co-processor has its own flat 256 MB; PAGE/HIMEM live there, far beyond the 64 KB view. On the Pi the co-processor is the same interpreter (or RunCPM's Z80, program 3) running on core 3 (core/tube_cp.c); an unfitted program leaves status reading 0. The console it talks to is JIM, the terminal at `$DA00` (core/term.h).

A.2 VICKY, SHEILA and the sprites

Generated from `core/vicky.h`.

VICKY

VICKY – the K4510 video chip.

Register block at IO_VICKY (`$D000`), 256 bytes, byte-addressed. Every pointer is a 28-bit physical address into main RAM; there is no video memory. Rendering is per scanline into an 8-bit indexed framebuffer the frontend supplies; the frontend applies vicky_palette_rgb().

`$00` **CTRL** bit0 display enable; the rest pick the mode. The glass is always 640x480 and the raster lines are always 0..479 – a smaller mode is drawn into it, doubled, and centred. bit1 columns halved (320), bit2 lines halved (240), bit3 a 200-line field (40 blank lines top and bottom), bit4 columns quartered (160; with bit1).

```
1      640x480       1|4     640x240
1|2    320x240       1|2|8   320x200
1|2|8|16  160x200    1|4|8   640x200
```

`$01` **BGCOL** background palette index (where nothing is drawn)

`$02` **RASTER** read: current line (low 8); write: raster-compare low

`$03` read: line high bits; write: compare high

$04	**IRQSTAT** bit0 vblank, bit1 raster==compare, bit2 SHEILA IRQ op, bit3 sprite collision. Write 1s to acknowledge.
$05	**IRQMASK** same bits; IRQ line = IRQSTAT & IRQMASK
$06	**PALIDX** palette index for the write port
$07	**PALR**
$08	**PALG**
$09	**PALB** – writing B commits the entry and increments PALIDX
$0A-$0D	**SPRTAB** 28-bit pointer to the sprite attribute table (128 x 16 B)
$0E	**SPRCTL** bit0 sprites enable
$0F	reserved
$60-$63	**SHEILA** 28-bit pointer to SHEILA's list
$64	**SHEILACTL** bit0 enable; the list restarts every frame at line 0
$70-$73	**BLTSRC** 28-bit
$74-$77	**BLTDST** 28-bit
$78,79	**BLTW** width px
$7A,7B	**BLTH** height px
$7C,7D	**BLTSS** source stride (bytes)
$7E,7F	**BLTDS** dest stride
$80	**BLTOP** 0 copy, 1 keyed copy (src 0 skipped), 2 fill (value = BLTSRC byte 0), 3 AND, 4 OR, 5 XOR, 6 LINE: from (LX0,LY0) to (LX1,LY1), colour = BLTSRC byte 0, into the BLTDST surface of stride BLTDS, clipped to 0..BLTW-1 x 0..BLTH-1 7 TRIANGLE: filled (LX0,LY0)-(LX1,LY1)-(LX2,LY2), same colour, surface and clip as LINE
$81	**BLTFLG** bit0 H-flip, bit1 V-flip
$82	**BLTCMD** write anything: go. Instant. Reads 0.
$84,85	**LX0**
$86,87	**LY0**
$88,89	**LX1**
$8A,8B	**LY1**
$8C,8D	**LX2**
$8E,8F	**LY2** (signed 16-bit) Blits are 8 bpp (one byte per pixel) in this version.
$90-$9F	**COLSS** read: sprite-sprite collision bits, one bit per sprite (sprite n hit another sprite this frame). Cleared on read of $40.

`$A0-$AF`	**COLSL** read: sprite-layer collision bits (sprite n over a non-transparent layer pixel). Cleared on read of `$50`.
	Sprite attribute entry, 16 bytes, in main RAM:
`+0,1`	X (signed 16)
`+2,3`	Y (signed 16)
`+4..7`	**DATA** 28-bit pointer
`+8`	**CTRL** bit0 enable, bit1 8 bpp (else 4), bit2 H-flip, bit3 V-flip, bits4-5 Z: drawn after layer Z (0..3)
`+9`	**SIZE** bits0-1 width 8/16/32/64, bits2-3 height 8/16/32/64
`+10`	**PALOFS** (4 bpp: index = PALOFS«4 | pixel)
`+11..15`	reserved
	Pixel 0 is transparent. No per-line limit. 128 sprites.

SHEILA – the display-list coprocessor (Doc named it, 2026-08-22; the Amiga's copper is the ancestor). 4-byte instructions in main RAM, executed at the start of each scanline until a WAIT blocks. Register writes take effect for the line about to be drawn.

`00`	**END** stop until next frame
`01`	**WAIT** lo hi wait for line >= (hi«8|lo)
`02`	**MOVE** reg val write val to VICKY register reg
`03`	**SKIP** lo hi skip next instruction if line >= value
`04`	**JUMP** a0 a1 a2 continue at 24-bit address
`05`	**IRQ** set IRQSTAT bit2
	At most 256 instructions per line are executed (runaway guard).
	Layers 0..3 at `$10` + n*`$10`, 16 bytes each:
`+0`	**LCTRL** bit0 enable, bits1-2 mode (0 bitmap, 1 tile, 2 text8, 3 text32), bits3-4 bpp (0=1, 1=2, 2=4, 3=8), bits5-6 cell size (tile: 8/16/32/64 px square; text: 0 = 8x8, 1 = 8x16)
`+1`	**LPALOFS** palette offset for <8 bpp: index = (value « depth) | pixel
`+2,3`	**SCROLLX** 16-bit, pixels
`+4,5`	**SCROLLY** 16-bit, pixels
`+6,7`	**STRIDE** bitmap: bytes per row. tile/text: map entries per row.
`+8..+B`	**DATA** 28-bit pointer: pixels (bitmap) or glyph/tile set
`+C..+F`	**MAP** 28-bit pointer: the map

Map formats:

`tile`	2 bytes/cell: bits 0-9 tile index, 10 H-flip, 11 V-flip, 12-15 palette offset (used for <8 bpp). Tile pixel data at DATA + index * (size*size*bpp/8), rows MSB-first packed.
`text8`	1 byte/cell: glyph index. 1 bpp 8xH glyphs at DATA + g*H. Colours from LPALOFS: index = (LPALOFS«1) | pixel.
`text32`	4 bytes/cell: glyph lo, glyph hi (16-bit index), fg, bg – byte-wide palette indices per cell. bit7 of glyph hi = reverse.

Layer 0 is bottom. Pixel index 0 is transparent in every layer; BGCOL is the ground (text32 bg is never transparent). Changed 2026-08-22 from "opaque in the lowest layer" so SHEILA backgrounds show under text.

A.3 The network

Generated from `core/net.h`.

The N: device

The network, three ways – each borrowed from the machine that did it first.

The Meatloaf rule (the C64's Meatloaf cartridge): a URL is a file name. A name beginning http://, https:// or tnfs:// that reaches the filesystem device (`$D300`) for reading – LOAD, TYPE, CP, RUN, EhBASIC's LOAD, BBC BASIC's LOAD on the Tube – is fetched and served like a file. No ROM change: the ROM passes names through untouched and the host fetches.

TNFS (the Spectranet's file protocol, the one FujiNet and Meatloaf servers speak; UDP, port 16384): a tnfs://host[:port]/path is also a DIRECTORY. CD tnfs://host/path puts the machine's current directory on the server: DIR lists it, a bare name loads from it (so the REXX rule runs programs off the internet), CD .. climbs it, CD - comes home. The server is read-only from here.

The N: device (FujiNet's network device, as the Atari and Apple saw it), at `$D900`, for programs that want a live connection: four channels, each a URL opened for reading and writing.

`$D900`	*W* command
`$D901`	*R* status (0 ok, 1 not found / refused, 2 i/o error, 3 bad command, 4 closed by the peer, 5 name too long, 6 not fitted)
`$D902`	*RW* channel 0-3
`$D904-$D907`	**NAMEPTR** 28-bit -> URL, NUL-terminated
`$D908-$D90B`	**ADDR** 28-bit

`$D90C-$D90F` **LEN** bytes requested; updated to bytes done

`$D910-$D913` **SIZE** after OPEN: the content length, `$FFFFFFFF` if unknown; after STATUS: bytes waiting

commands: 1 OPEN (tcp://host:port a connection; http://, https://, tnfs:// a file, fetched whole, then READ)

2 READ (up to LEN bytes to ADDR, what has arrived so far; LEN = done, 0 = nothing yet)
3 WRITE (LEN bytes from ADDR; tcp only)
4 CLOSE
5 STATUS (SIZE = bytes waiting; status 4 once the peer has closed and the bytes are gone)
6 GET (the Meatloaf rule for programs: fetch the whole URL into ADDR, at most LEN; LEN = done, SIZE = total)

Reads never block the machine: a program polls, as it would a UART. Underneath: core/net_plat.h – sockets and curl on the desktop, Circle's stack on the Pi (no TLS there: https answers 6).

A.4 JIM, the terminal

Generated from `core/term.h`.

JIM

JIM, the terminal (`$DA00`) – the Beeb's third page, given a job: a VT100 with the ANSI colour and VT220 editing additions, in hardware – the way a real 8-bit machine got a serious terminal: a card, not a program. It draws on the VICKY text32 screen the ROM console uses, inside the geometry the ROM gives it, so the console and the terminal share one screen and one cursor. Anything that needs a terminal writes its byte stream here: the ROM for the Tube (CP/M programs set up for VT100/ANSI, BBC BASIC's console edition) and TELNET for the BBSes (ANSI-BBS: CP437 glyphs, 16 colours).

`$DA00` *W* **DATA** a byte of the stream

`$DA01` *R* **STATUS** bit7 a reply byte waits; bit0 the stream moved the cursor since CX/CY were written

`$DA02` *R* **REPLY** the next reply byte (pops): answers to ESC[6n / ESC[c, and translated keys

`$DA03` *W* **KEY** a K4510 key code (io.h): its terminal bytes go to REPLY (arrows ESC[A.. or ESC OA.. in application mode,

	Home/End, PgUp/PgDn/Ins ESC[n~, Del $7F, F1-F4 ESC OP.., F5-F12 ESC[15~.., everything else through unchanged)
$DA04	*W* **CTRL** 1 reset (modes, attributes, cursor home; the screen kept) 2 clear the screen and home
$DA05–$DA0D	*RW* **COLS ROWS OX OY CX CY FG BG STRIDE** the window: origin (OX,OY) cells, STRIDE cells per row
$DA0E	*RW* **FLAGS** bit0 cursor shown (blinking) bit1 read: application cursor keys (DECCKM)
$DA10–$DA13	*RW* **BASE** 28-bit address of the text32 map (reset: $030000)
$DA14,$DA15	*RW* **DEFFG DEFBG** the colours SGR 0 / 39 / 49 return to

Sequences: the VT100 set (cursor, ED/EL, DECSTBM, DECSC/DECRC, IND/RI/NEL, tabs, DECAWM/DECOM/DECCKM, DEC line drawing via ESC(0 and SO/SI, DSR, DA, DECALN, RIS), ANSI SGR 0/1/4/5/7/22/24/27/30-37/39/40-47/49/90-97/100-107 and 38;5;n / 48;5;n for n < 16, VT220 ICH/DCH/IL/DL/ECH/SU/SD/CHA/VPA, IRM, ESC[?25 cursor, ESC[s/u, DECSTR. Bytes $80–$FF are glyphs (CP437).

A.5 Memory and the far view

Generated from core/mem.h.

The ROM window and the stub page

The ROM image lives in the top 64 KB of physical memory and is seen in the unmapped CPU view from mem_rom_base up; the physical RAM at $A000–$FFFF is "RAM under the ROM", revealed by banking blocks 5/7 onto $A000/$E000 (K-05). The page $FF00–$FFFF always reads the ROM, whatever is banked: the system-call stub and the vectors live there.

Chapter B

Filing an Issue

This machine has one author, and an issue you file will, in all likelihood, be handed straight to an AI coding session to work on, as written. That is worth knowing, because it changes what a good report is: not a description of the fault, but the machine's own account of it. The machine can write most of that account itself, and the whole of this appendix is how to let it.

B.1 Before you try again

None of these is politeness. Each one removes a day.

Turn `DUMP ON` before you reproduce it. Then every fifteen seconds, and again the instant you type `DUMP` once it has gone wrong, the machine writes its whole state — registers, the screen as you see it, the last 4096 instructions, your keystrokes, and the log of every command you have typed — to `dumps/dump-NNN.txt` on the host. A dump taken while it is wrong is worth more than any description either of us could write, and `DUMP ON` means you cannot miss the moment.

Try it once with `k4510 --no-startup.bat`. A `/STARTUP.BAT` that has gone wrong looks exactly like a machine that has gone wrong, and telling them apart has cost this project a day before. Ten seconds of yours settles it.

Check this book. If the machine disagrees with a page here, that is still a fault worth filing — it is this book's, and it is fixed the same way. Say which page.

B.2 BUG writes the report

Then let the machine do the writing:

```
/] BUG
```

`BUG` interviews you — seven questions, one at a time, each answered in a line or two — and writes the finished report to

```
/SYSTEM/BUGREPORTS/BUG-YYYYMMDD-HHMMSS.TXT
```

making the directory if it is not there. Five lines it fills in itself, and they are the five most often got wrong by hand: which machine (the desktop emulator, or bare metal on a Pi), the exact build (`INFO -v` says the same — a trailing `+` means the emulator was built from an edited tree, which is exactly the report that will not reproduce), the screen you are in, the last dump, and when.

The questions are the ones a coding session would otherwise have to ask you: where you were (the shell, a BASIC, CP/M…), what you typed *exactly*, what happened, what you expected instead, and *why* you expected it — the line most reports leave out. The K4510 is a fantasy machine (page 67): there is no silicon it can be measured against, so "wrong" only means something next to what you were expecting and where that came from. It is also what separates a bug from a design decision from an error in this book, which are three different repairs.

`BUG` is a program rather than a command, so it reaches you everywhere the * escape does: `*BUG` from either BASIC, from the monitor, from CP/M.

`DUMP` first, then `BUG`. If the report's `Dump` line says there is none, the more valuable half is missing. Type `DUMP` while it is still wrong and run `BUG` again — the two travel together, and this is the one thing that is easy to do in the wrong order.

B.3 Where it goes

```
https://github.com/mlongval/bmc-k4510/issues
```

From the host, the report is in `fs/SYSTEM/BUGREPORTS/` and the dump in `dumps/`: open an issue, attach both, and you are done — the report is the whole text of the issue. The form there asks `BUG`'s questions again for anyone who has no report file: a fault in this book, a machine that

will not boot, a telephone. If you have no account there, the same two files in an email are worth exactly as much.

Feature requests and disagreements are welcome in the same shape — "what I expected" does the work whether or not anything is broken.

One thing to check before you attach a dump. It carries the shell log, which is every command line you typed in that session, and file names from your own disk along with them. It is a plain text file: open it and have a look before it goes anywhere public.

Chapter C

The Sound, and What It Took

Every other chapter of this book describes the machine as it is. This one describes how it got that way, because the sound took longer to get right than anything else in it and the route was instructive. Nothing here is needed to use the machine. It is here because a fantasy computer is a series of decisions, and the decisions about sound were the ones that most often turned out to be about something else.

It has one thesis, and it is worth stating before the evidence rather than after: **the sound was hard because keeping a real-time audio ring fed from a machine that runs in bursts is hard, not because emulating a SID is expensive.** Every fault the sound reported turned out to be somewhere else, and the chip emulation — the part everyone suspects — has never once been on the critical path.

If you are reading it to find out whether the machine keeps its four SID chips, look at the last section: at the time of this printing that is an open question, and it is not the question this appendix is mostly about.

C.1 Four chips, because it is a fantasy

The C64 had one SID. The plan for this machine was four, at `$D400` and every `$20` after it, for no better reason than that a machine which never existed may have whatever its author thinks it should have and four is a chord. Dag Lem's reSID — the emulation that has been the reference for thirty years — was vendored unchanged from VICE 3.3, wrapped in `core/sid.cc`, and mixed to 16-bit mono. The frontend fed a ring buffer a scanline at a time and an audio callback drained it at 48 kHz.

That took an afternoon and it worked. Everything after this section is the six weeks of it not quite working.

C.2 A SID is not a CPU

The first mistake was in the first version and is the kind only a fantasy machine can make: the SIDs were clocked at the CPU's clock.

On a C64 the two are the same crystal, so the question never comes up.

Here the CPU runs at 40.5 MHz and the SID is a 1 MHz part, and clocking reSID at 40.5 million cycles a second cost **21 milliseconds a frame** — more than the whole frame — and put every pitch out by a factor of forty-one. The fix was one line and the lesson outlived it: *the SID's clock is not the CPU's*, and every later change that touched the clock had to be told so again. The most recent time was in August 2026, in a benchmark that swept the CPU clock and forgot to tell reSID, and rendered every note at the wrong rate.

C.3 "Choppy choppy choppy" — which was not the sound

The machine's first outing on real hardware, a Raspberry Pi 3B+ on a television, produced the report that named the problem for the rest of the project: *the SID sounds on the RPi3 are very very bad, choppy choppy choppy*.

It was not an audio bug. The Pi was running the machine at about 17 ms against a 16.7 ms frame, and two recent changes had spent that slack — a debug recorder storing every instruction fetch, and a rule that made the I/O page win on every memory access, which put an extra comparison on every read and write in the machine. Neither was visible on a desktop. Both were fatal on the Pi, and what they broke was the sound, because the sound is the first thing that breaks when a frame runs long.

That is the pattern the rest of this appendix is about. **The sound is the machine's alarm bell.** Almost every time it rang, the fault was somewhere else.

C.4 Silence is not free

With a clean power supply and honest measurements, the Pi at 20 MHz sat 1.6 ms over its frame budget. Of the 18.3 ms, **3.4 was the SIDs** — all four of them, three of them silent.

reSID costs the same for silence as for sound: a chip that has been written to is a chip that must be clocked, whether or not anything is audible. So a chip is now clocked and mixed only once the machine has written to it since reset, which at the prompt is three chips' worth of frame given back. Note what that 3.4 ms was — four chips *clocked*, not four chips playing — and that it is the largest figure the sound has ever cost anywhere in this project.

That fix has a tail which is still in the machine: the cost is latched by the *first write*, not by whether anything is sounding. A program that pokes a SID register during setup and then goes quiet pays the full price for the rest of the session. On a desktop that is half a millisecond of sixteen and nobody notices. On a Pi with no slack it is worth knowing.

C.5 The ring that started empty

On the Pi the audio callback is served by the same core that runs the machine, once a frame, a block at a time. A frame makes 800 samples; the block was 1024; the ring started empty. So the ring's level lived between nothing and one frame, and a callback regularly found 800 real samples and 224 of silence — a gap, on a steady beat, at a frame rate the machine was comfortably holding.

Smaller blocks and a lead of three of them — 32 ms, which a music player does not feel — and the level never touches the floor.

It made things worse. At 15 MHz the gap count went from 40–48 in two seconds to 52–62.

C.6 Chips that play on without the machine

The lead failed because the model behind it was wrong, and the numbers said so plainly. At 15 MHz the Pi ran at 58–60 frames a second, not a locked 60. Audio was made only by the machine's frames, 800 samples

each. At 58 frames that is 46,400 samples a second against the 48,000 the device consumes. *Any* shortfall drains the ring and then gaps for ever after; a lead only postpones the first one. Even rounding the cycles per scanline down from 520.83 to 520 is a 0.16 % deficit on its own.

So the machine stopped tying its sound to its frames. Real hardware does not stop its sound chips because the processor stalled — a SID keeps oscillating whatever the 6502 is doing — and now nor does this one: after a frame, if the ring is below its target, the SIDs are clocked on *without the CPU* until it is full. The SID clock does not move, so pitch does not move; a slow frame sustains a note a fraction longer rather than cutting it.

This is the most important decision in the whole story, and it is the one that came back.

C.7 The meter that read zero

With the chips decoupled, and with the Pi's callback served every 128 lines rather than once a frame, the gap counter read **zero**, on three boots, at the default clock. By every instrument the project had, the sound was clean.

Doc: *both say 0 drops but I still can hear them.*

He was right and the meter was wrong. The gap counter counts callbacks that found the ring *empty* — it counts silence. And the decoupling exists precisely to ensure the ring is never empty. So across the entire band where a host is merely losing rather than drowning, the counter reads zero and there is nothing wrong with the sound except that a growing fraction of it was not made by the program. The SIDs were playing on without the machine. That is what he was hearing.

It is now counted. Beside the gaps there is a second number: how many samples the machine did not make and the chips supplied. On one host, at rows both instruments had called clean:

Clock	Frames	Gaps	Filled in
30 MHz	53 fps	0	36,464
20 MHz	60 fps	0	16,818
10 MHz	60 fps	0	1,609

of 96,000 samples in the two seconds measured. At 30 MHz *more than a third of the sound was not the program's*, and every meter said clean.

Clean now means three things at once, because any two of them can be true while the sound is wrong: sixty frames a second, no gaps, and under two per cent filled in.

The moral, three times over. A performance regression that presented as an audio bug. An audio fix that made the gap count worse. A gap counter that read zero because an earlier fix had guaranteed it would. Each time, the instrument was measuring something adjacent to the fault — and each time, the person listening to the machine was right before the numbers were.

C.8 A machine keeping somebody else's time

The newest finding, and the one that reframes the rest: on the desktop the machine was not keeping its own time at all.

The Pi has always paced itself against the wall clock, because its graphics layer blocks until the display flips and a frame that overran by a millisecond would wait for the next one and halve the frame rate. The desktop did the opposite — it left the pacing to the display's vertical sync and did none of its own, so the machine ran at whatever the compositor handed it. On one host, a 60 Hz display, that was **51.8 frames a second**, with the act of putting the picture on the glass taking 16.9 ms of a 19.3 ms frame.

A machine at 51.8 frames a second makes 51.8 frames of sound where the device wants 60. The chips filled in the missing seventh, exactly as designed, and it was audible — and the new counter measured it at 17 %. The machine now keeps its own 60 Hz on the desktop as it always has on the Pi, and presents when it is ready:

	Whole frame	Onto the glass
before	19.294 ms (51.8 fps)	16.867 ms
after	16.668 ms (60.0 fps)	0.260 ms

C.9 Where it stands

At the time of this printing the machine has four reSID chips in it, and whether it keeps them is an open question — but not the question it first looked like.

It was raised as a performance question, after a Raspberry Pi that would hold only 15 MHz. On that ground it is settled, and against replacement. **reSID is not the cost.** One chip sounding measures **0.039 ms of a 16.67 ms frame** on a desktop — a quarter of one per cent — and four chips written since reset come to about half a millisecond. The largest the sound has ever cost is the 3.4 ms of §C.4, on the Pi, before silent chips stopped being clocked, and that was a bookkeeping fault rather than a fidelity one. Nothing measurable is bought by removing them. (What four *sounding* chips cost on the Pi has never been measured — the card is bare metal and the desktop figure does not transfer to an A53. If it were half a millisecond it would be some three per cent of that frame, which would still not be the bottleneck; but that is an estimate, and it is flagged as one.)

Where the question is real is identity, and there it deserves both sides. The SID is a part this machine inherited rather than chose: it is in here because the C64 had one, in the same way the * prefix is in here because Acorn had it. A machine that never existed is entitled to a voice its author picked on purpose — and the original specification for this one read *4 SIDs/OPL2*, so a synthesiser of the other tradition would not be a betrayal of the plan but the rest of it. Against that: four SIDs are what the machine has sounded like since its first afternoon, they cost almost nothing, and the entire body of C64 music plays on them, which is not a small thing to hand back.

So the honest statement is that the chips are cheap, the case for changing them is about what the machine wants to be rather than what it can afford, and that case has not been decided. If it is, this appendix will say what replaced them and why, and will keep everything above it — because the route matters more than the destination, and a book that quietly rewrote its own history would be worth less than one that admits what it tried.

Disclaimer

The BMC-K4510 is a hobby project: a computer that does not exist, built for the pleasure of building it. It is offered to you as a gift, and it comes with exactly the guarantees a gift comes with — none.

In the words of the licence it is distributed under (page 74), and meaning every one of them:

> **THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW.** EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.
> **IN NO EVENT** WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM.

In plainer words: nobody promises this machine works, that it will keep working, or that anything it writes to your disk is safe. Keep backups. Do not fly aeroplanes with it, run hospitals on it, or trust it with money. It is a toy, and it is allowed to be wrong.

The same goes for this book: where it and the machine disagree, the machine is right. Nothing here is endorsed by or affiliated with Acorn, Commodore, Microsoft, Digital Research, the MEGA65 project, the Raspberry Pi Foundation, or anyone else gratefully named in these pages. The mistakes are the author's alone.

Constructive comments can be left at the repository's issues page:
`https://github.com/mlongval/bmc-k4510/issues`

All complaints, criticisms and negativity can be addressed to
`/dev/null`

Thank You

The BMC-K4510 is a machine that never existed. Almost everything in it that *does* exist was written by somebody else first, and given away. This chapter is the thanks; the Licences chapter (page 74) is the paperwork.

The heart of the machine

Gábor Lénárt (LGB) wrote the 45GS02 CPU core in Xemu, the MEGA65 emulator. It runs here unchanged — not a line altered, not a bug fixed — and every instruction this book describes is his code executing. There would be no K4510 without it.

Dag Lem wrote reSID, the SID emulation that has been the reference for thirty years. The K4510's four sound chips are four copies of it (the 6581 model, as shipped in VICE 3.3).

The VICE team, whose decades of emulation scholarship — documentation, test programs, arguments settled in code — this project consulted at nearly every turn.

The tongues

Lee Davison (1966–2013) wrote EhBASIC, the machine's first BASIC and still the one that comes up in ROM. He is not here to be asked about the graphics and sound words bolted onto it; the machine carries his name in its `README` and its startup banner instead.

Richard T. Russell wrote BBC BASIC, and has kept writing it for forty years. The console edition of his BBC BASIC for SDL 2.0 (BBCTTY) is what runs on the Tube co-processor in Chapter 4, under the zlib licence he publishes it with. "BBC BASIC" is the name of his interpreter; this book uses it only to say what is running, and claims nothing in it.

Marcelo Dantas (“Mockba the Borg”) wrote RunCPM, which is the whole of Chapter 6: a complete CP/M 2.2 with its own CCP, vendored here unmodified and simply handed a Z80’s worth of address space.

Scot W. Stevenson, **Sam Colwell** and **Patrick Surry** wrote Tali Forth 2 and put it in the public domain — a Forth written to be read, which is why Chapter 5 can be honest about how it works.

Tomasz Biela (“tebe”) wrote Mad Pascal and MADS; the K4510 target in `pascal/` is a guest in his compiler. **Wojciech Bociański** (“bocianu”), whose Neo6502 target showed how such a guest should behave.

Steve Wozniak wrote Wozmon in 1976 in 256 bytes, and the monitor in this machine is still recognisably it.

Michael Steil maintains msbasic, and **Microsoft** released 6502 BASIC 1.1 under the MIT licence in 2025 — between them, the machine’s next native BASIC.

Letters on the screen

Fonts are the part of a computer you look at longest, and clean ones with a clear pedigree are hard to come by.

The Linux kernel contributors — the 8x8 console font (`font_8x8.c`) that got the text mode on its feet and is still the default.

Paul Gardner-Stephen and **Roman Standzikowski** (FeralChild64) — the clean-room C64/C65 chargen from MEGA65 open-roms, and **Retrofan**, whose PXLfont travels with it by permission.

Ville-Matias Heikkilä (“Viznut”) — unscii, placed in the public domain.

Damian Vila — BESCII, the PETSCII spirit with a clean pedigree, CC0.

The bare metal

Randy Rossi wrote BMC64: VICE on a Raspberry Pi with no operating system under it, 50 Hz smooth scrolling and input latency measured in single frames. It is the direct reason this machine has a Pi port at all

— the route to the Pi was always meant to be BMC64's `emux_api` seam, and the bare-metal case it proved is what made the port thinkable. He also built the **VIC-II Kawari**, a modern drop-in VIC-II with video modes the original never had: the demonstration that you may extend an 8-bit machine's video chip and still be working inside the tradition rather than outside it. VICKY is a more reckless cousin of that idea. **minch** (aminch) has taken BMC64 over from him and carries it onto the newer Pis; the project is in good hands.

Rene Stange wrote Circle, the bare-metal C++ environment that is the entire reason a Raspberry Pi 3B+ can boot straight into this machine with no operating system under it. **Xalior** wrote circle-libsdl2, which let the desktop emulator move to the Pi essentially as written.

The workshop

The machine is built with **cc65** (compiler, assembler, linker and runtime for the whole system ROM), **64tass**, **NASM**, **GCC** and **GNU Make**; it shows itself to you through **SDL2** — Sam Lantinga's library and its contributors', which is the window, the keyboard and the sound on the desktop, and through Xalior's port, on the Pi as well. This book is set in **XeLaTeX** with **Clear Sans** and **Iosevka**, and every screenshot in it was captured from the machine actually running — a picture that cannot be produced fails the build.

Heritage

Some debts are not code.

Acorn Computers — the Tube, the sideways-ROM model, the * command prefix, and the idea that a second processor should be a normal thing to own. The Beeb's ghost is all over this machine.

Commodore — the other half of its soul: PETSCII, the SID, and the line from the PET to the C65 that stopped one machine short of this one.

The MEGA65 project — for keeping the 45GS02 and the C65 dream alive in real silicon, and for open-roms.

Kim Lemon and the Lemoners — Lemon64 has been the C64's front door since 1998: the games database, the reviews and scans, the music, and a forum that actually answers. Twenty-eight years of one person's dedication to a machine's community, and a good half of the small facts this project needed came from there.

The Meatloaf project (idolpx) — the C64 cartridge that made a URL a file name, which is the whole of this machine's rule for the network: name it, and it is fetched. **The FujiNet project** — the network device the Atari got first, whose N: device this machine imitates and whose TNFS servers it speaks to; `CD tnfs://fujinet.online/` is a directory on theirs.

Paul Scott Robson wrote the Neo6502's firmware — the operating system and BASIC that make an Olimex Neo6502 a computer rather than a board — and the handbook that documents it. That book is the one this one is modelled on, down to the page size, and the rule that every example must be run and photographed before it may be printed is his. The other model is the **MEGA65 User's Guide**.

The High Voltage SID Collection and the composers in it, whose tunes are what `SIDPLAY` plays on a good evening. None of that music is distributed with this machine (see the Licences chapter); it is theirs.

The project's orchestrator is Michael Longval — Doc — who decided what the machine is, what it borrows and what it refuses, what gets built next and what gets thrown away, and read every page of this book against the machine.

And the machine's other author. Most of the K4510's own code — VICKY, SHEILA, the DMA engine, the ROM, K/OS, the Tube ULA, the editors, the Pi port — was written in conversation with Claude (Anthropic's Claude Code), over several months of long sessions with a build log to prove it. The design decisions are the author's; a great deal of the typing was not.

And whoever is missing

This list was assembled by hand and is certainly incomplete. If your

work is in this machine and your name is not on this page, that is an error and not a judgement — open an issue at `https://github.com/mlongval/bmc-k4510` and it will be fixed in the next printing.

Licences

The Thank You chapter (page 69) is the thanks; this is the record. The authoritative copy is `LICENSES.md` in the repository — if the two ever disagree, that file wins.

The project itself

The BMC-K4510 as a whole is distributed under the **GNU General Public License, version 2 or (at your option) any later version**. Everything written for this project — the machine, VICKY, SHEILA, the DMA engine, the MATH unit, the system ROM and K/OS, JIM, the Tube ULA, the network layer, the editors, the demos, the Pascal target and the Raspberry Pi port — is Copyright © 2026 Michael Longval and is released under those terms. The choice is not really a choice: the CPU core and reSID are GPL-2.0-or-later, and a work containing them must be too.

Code inside the machine

- **Xemu 65xx/45GS02 CPU core** — Gábor Lénárt. `core/xemu/`: `cpu65.c`, `cpu65.h`, `cpu65_mega65_timings.h`, `cpu65ce02_disasm_tables.c`, used unchanged. *GPL-2.0-or-later.*
- **reSID** — Dag Lem, as shipped in VICE 3.3. `core/resid/`. *GPL-2.0-or-later.*
- **BBC BASIC for SDL 2.0, console edition (BBCTTY)** — Richard T. Russell. `tube/`, vendored and altered (see `tube/ALTERED.md`). *zlib licence.* "BBC BASIC" is the name of Mr Russell's interpreter, which he publishes under his own arrangement with the BBC. This project has not sought, and does not hold, any licence to the name: it is used here only to identify the interpreter, which is why this book says "Richard Russell's BBC BASIC" and not "the K4510's".

- **EhBASIC 2.22** — Lee Davison, in the ca65 form from jefftranter/6502. `basic/basic.asm`. *Free for non-commercial use only*; derivatives must carry the words "Derived from EhBASIC" — see `basic/README-EhBASIC.txt`. It ships as a separate program (`fs/ehbasic.prg`) and is not linked with the GPL code.
- **RunCPM** — Marcelo Dantas. `cpm/src/`, vendored unmodified (`cpm/VENDORED-FROM.txt`). *MIT.*
- **Tali Forth 2** — Scot W. Stevenson, Sam Colwell, Patrick Surry. `forth/tali/`, vendored unmodified. *Public domain.*
- **Wozmon** — Steve Wozniak's 1976 monitor, reimplemented here for the 45GS10. Apple published the original listing; this machine's version is the project's own code under the project licence.
- **cc65 runtime** — the ROM and every `.prg` are linked against `none.lib`. *cc65's zlib-style licence.*
- **Microsoft 6502 BASIC 1.1** (planned, via mist64/msbasic) — *MIT*, released by Microsoft in 2025. Not yet in the machine.

Fonts

- **Linux kernel 8x8 console font** — `data/font8.bin`, built by `data/mkfont.py` from `lib/fonts/font_8x8.c`. *GPL-2.0.* This is the default text font.
- **MEGA65 open-roms chargen + PXLfont 2.3** — `data/fonts/openroms/`. Clean-room C64/C65 character shapes; PXLfont is Retrofan's, included with open-roms' recorded permission. *LGPL-3.0-or-later* — `8x8font.png` is shipped as the editable source, for LGPL compliance.
- **unscii-8** — Ville-Matias Heikkilä. `data/fonts/unscii/`; `font8-unscii.bin` is a generated drop-in alternative to `data/font8.bin`. *Public domain.*
- **BESCII v3** (Mono + source) — Damian Vila. `data/fonts/bescii/`. *CC0-1.0.*

- **Clear Sans** (Intel, *Apache-2.0*) and **Iosevka** (Belleve Invis, *SIL OFL 1.1*) — the two faces this book is set in. They are not part of the machine.

No Commodore ROMs here. Until 2026-08-23 the text font was derived from a Commodore 64 character ROM. That file and every derivative were removed from the repository *and from its history* before it was made public. If you own a C64 and want the real thing, drop your own `chargen.bin` into `/SYSTEM`: the machine will use it, and `.gitignore` will keep it out of the repository. That copy is yours, not ours to publish.

The Raspberry Pi build

Neither of these lives in the repository; `pi/Makefile` expects them beside the checkout.

- **Circle** — Rene Stange. *GPL-3.0.*
- **circle-libsdl2** — Xalior. *zlib*; its `sdl-app.ld` is *GPL-3.0*.

A `kernel8.img` built from these is therefore GPL-3.0 as a whole, which GPL-2.0-*or-later* code permits. The Raspberry Pi boot firmware in a distributed card image (`bootcode.bin`, `start.elf`, `fixup.dat`) is Broadcom's, redistributable with Raspberry Pi hardware under its own licence.

Build tools

Not distributed with the machine, but required to build it: GCC and GNU Make (*GPL*), cc65 (*zlib-style*), 64tass (*GPL-2.0*), NASM (*BSD-2-Clause*), SDL2 (*zlib*), Python 3 (*PSF*), and for this book XeLaTeX (*LPPL*). Mad Pascal and MADS (Tomasz Biela, *MIT*) are needed only if you compile Pascal; the K4510 target in `pascal/` is installed into your own Mad Pascal checkout.

What is deliberately not shipped

- **The CP/M system disk.** RunCPM's `DISK/A0.zip` — Digital Research's `ASM`, `MAC`, `DDT`, `STAT`, `SUBMIT` and third-party tools — installs into `fs/CPM/A/0/` for your use but is not committed: its files have mixed provenance and this repository is public. The same goes for WordStar, Turbo Pascal 3 and MBASIC, which the `K-*.SUB` launchers know how to start but which you must supply yourself.
- **SID tunes.** `fs/SID` is a symbolic link to a local copy of the High Voltage SID Collection. The tunes are the composers' copyright, are used here only for listening, and are not part of this repository or of any release.
- **A Commodore character ROM** — see the note above.
- **Your `STARTUP.BAT`** — yours, not the repository's (copy `/SYSTEM/STARTUP.SAMPLE` to start one).

If you redistribute the machine

Ship the sources you built from, keep `LICENSES.md` and this chapter with them, keep `8x8font.png` beside the open-roms font, and remember two things that are easy to forget: EhBASIC is non-commercial-only, and "BBC BASIC" is the name of Richard Russell's interpreter rather than of anything made here — describe it as his, as this book does, and give whatever you build on it a name of its own.